❖ 出现Belady现象的原因：

* FIFO算法的置换特征与进程访问内存的动态特征是矛盾的。
* 即被置换的页面并不是进程不会访问的。
* 也即是说各个页面的调入次序不能准确反映页面的访问情况，因为先进入内存的某些页面，可能是今后经常要访问的页面。

操作系统考试复习

操作系统作用



1.1.2 操作系统的作用(2)

❖ 2. OS作为计算机系统资源的管理者

- 处理机管理：用于分配和控制处理机
- 存储器管理：主要负责分配及回收内存
- I/O设备管理：负责I/O设备的分配(回收)与操纵
- 文件管理：实现文件的存取、共享和保护。

进程

定义

❖ 1. 进程的特征和定义

* 1)、较典型的定义：
  - (1)程序的一次执行过程。
  - (2)进程是一个程序及其数据在处理机上顺序执行时所发生的活动。
  - (3)进程是程序在一个数据集合上运行的过程，它是系统进行资源分配和调度的一个独立单位。

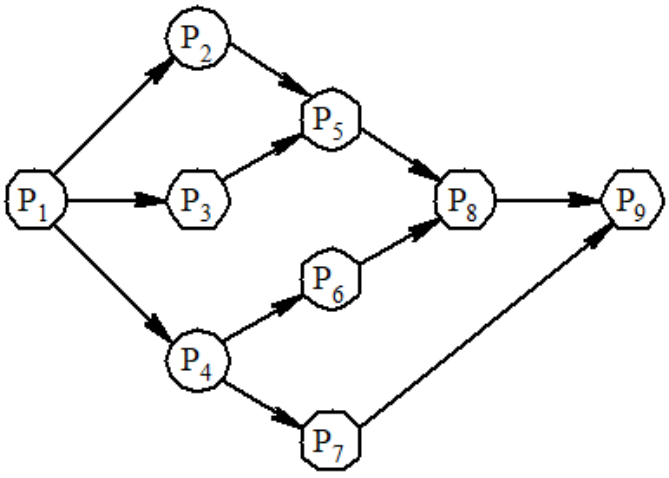
❖ 在引入了进程实体的概念后，我们可以把传统OS中的进程定义为：“进程是进程实体的运行过程，是系统进行资源分配和调度的一个独立单位”。

特征

1. 进程的特征和定义

* 2)、特征：
  - (2). 并发性
    - 只有建立了进程，才能并发执行。
  - (3). 独立性。
    - 独立运行，独立获得资源，独立接受调度。
  - (4). 异步性：（间断性）
    - 进程按各自独立的、不可预知的速度向前推进。

前驱图



(a) 具有九个结点的前趋图



(b) 具有循环的前趋图

图 2-2 前趋图

三个基本状态（必考）

❖ 2. 进程的三种基本状态

- 就绪状态
- 执行状态
- 阻塞状态

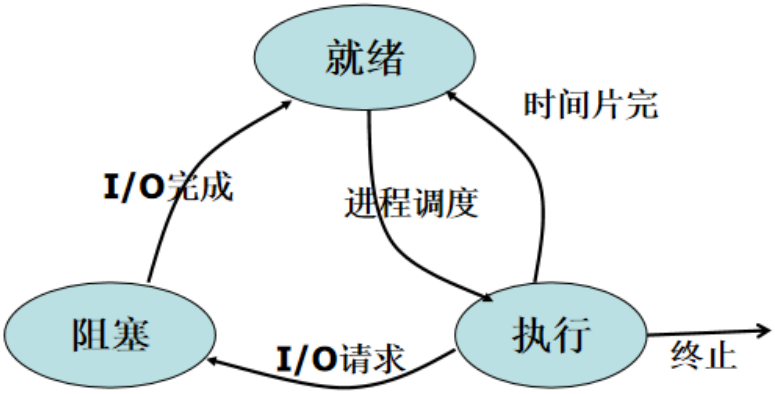


图2—5 进程的三种基本状态及其转换



进程控制块

❖ 1. 进程控制块的作用

- 是进程存在的唯一标志；
- 作用：使一个在多道程序环境下不能独立运行的程序（含数据），成为一个能独立运行的基本单位，一个能与其它进程并发执行的进程。
- 或者说，OS是根据PCB来对并发执行的进程进行控制和管理的。
- PCB(process control block)常驻内存

❖ 1. 进程控制块的作用

- （1）作为独立运行基本单位的标志。
- （2）能实现间断性运行方式。
- （3）提供进程管理所需要的信息。
- （4）提供进程调度所需要的信息。
- （5）实现与其它进程的同步与通信。

处理机状态（处理机上下文）

2）处理机状态

处理机状态信息也称为处理机的上下文，主要是由处理机的各种寄存器中的内容组成的。

进程同步

多道程序系统中进程是并发执行的，这些进程之间存在着不同的相互制约关系，为了协调进程之间的相互制约关系，就需要实现进程的同步。

互斥是同步的一种特殊情况。

临界资源（代码，题）

❖ 2. 临界资源：（一次仅允许一个进程访问的资源）

- 引起不可再现性是因为临界资源没有互斥访问。

临界区（代码，题）

3. 临界区

❖ 定义：进程访问临界资源的那段代码。

❖ 访问临界资源的描述：

进入区：检查有无进程进入

临界区：

退出区：将访问标志复位

可把一个访问临界资源的循环进程描述如下：

```
while(TURE){
        Entry section
          Critical section
        Exit section
         remainder section;
}
```

同步机制的准则

4. 同步机制应遵循的准则

- 1. 空闲让进：当无进程处于临界区时，允许一个进程进入自己的临界区。
- 2. 忙则等待：当已有进程进入临界区时，其它试图进入临界区的进程必须等待。
- 3. 有限等待：应保证为有限等待，不会产生死等。
- 4. 让权等待：不能进入临界区的执行进程应放弃CPU执行权。

信号量（大题）

是什么

> - 1 整型信号量
>   - 最初由Dijkstra把整型信号量定义整型信号量为一个用于表示资源数目的整型量S。
>   - 它与一般整型量不同，除初始化外，仅能通过两个原子操作wait(S)和signal(S)来访问。很长时间以来，这两个操作一直被分别称为P、V操作。

wait取资源，signal放资源

- 1 整型信号量
  - wait和signal描述如下：

```
wait(S){
        while(S≤0);    /* do no-op */
        S--;
}

signal(S){
        S++;
}
```

- 在整型信号量机制中的wait操作，只要是信号量S≤0，就会不断地测试。该机制不满足“让权等待”的准则，而是使进程处于“忙等”的状态。
- 记录型信号量机制是一种不存在“忙等”现象的进程同步机制。

整形信号量会进入忙等，记录型信号量不存在这个问题

记录型信号量的描述如下：

```
typedef struct{
    int value;
    struct process_control_block *list;
}semaphore;
```

- 其中整型变量value用于表示资源数目。
- 进程链表指针list，用于链接等待访问同一临界资源的所有进程。

这时wait(S)和signal(S)的描述如下：

```
wait(semaphore *S){
    S->value--;
    if(S->value <0) block(S->list);
}

signal (semaphore *S){
    S->value++;
    if( S->value<=0) wakeup(S->list);
}
```

- 用wait(S)和signal(S)实现同步与互斥。

在记录型信号量机制中：

- S->value初值：表示系统中某类资源的数目，故又称为资源信号量。
- 如果S->value的初值为1，表示只允许一个进程访问临界资源，此时的信号量转化为互斥信号量。
- S->value<0时，其绝对值表示该信号量链表中已阻塞进程的数目。

3 AND型信号量

- 死锁：在无外力作用下的一种僵持状态。
- AND同步机制的基本思想：将进程在整个运行过程中需要的所有资源，一次性全部地分配给进程，待进程使用完后再一起释放。
- AND同步机制特点：要么全分配，要么一个也不分配。
- 为此，在wait操作中，增加了一个“AND”条件，故称为AND同步，或称为同时wait操作，即Swait(Simultaneous wait)定义如下：

3 AND型信号量

```
Swait(S1, S2,…, Sn)
{
  while(TRUE)
  {
    if( S1≥1 && …&& Sn ≥1){
       for(i=1;i<=n;i++) Si--;
       break;
    }
    else {
       place the process in the waiting queue with the first Si found
       with Si<1, and set the program count of this process to the
       beginning of Swait operation
    }
  }
}
```

```
Ssignal(S1,S2,…,Sn) {
    while(TRUE) {
        for( i=1;i<=n;i++) {
            Si++;
            remove all the process waiting in the queue associated
             with Si into the ready queue
        }
    }
}
```

应用

2.3.4 信号量的应用

❖ 1. 利用信号量实现互斥

* 利用信号量实现进程互斥的进程可描述如下：

```
semaphore mutex =1;
  PA(){
      while(1) {
          wait(mutex);
          critical section
          signal(mutex);
          remainder section
      }
  }
```



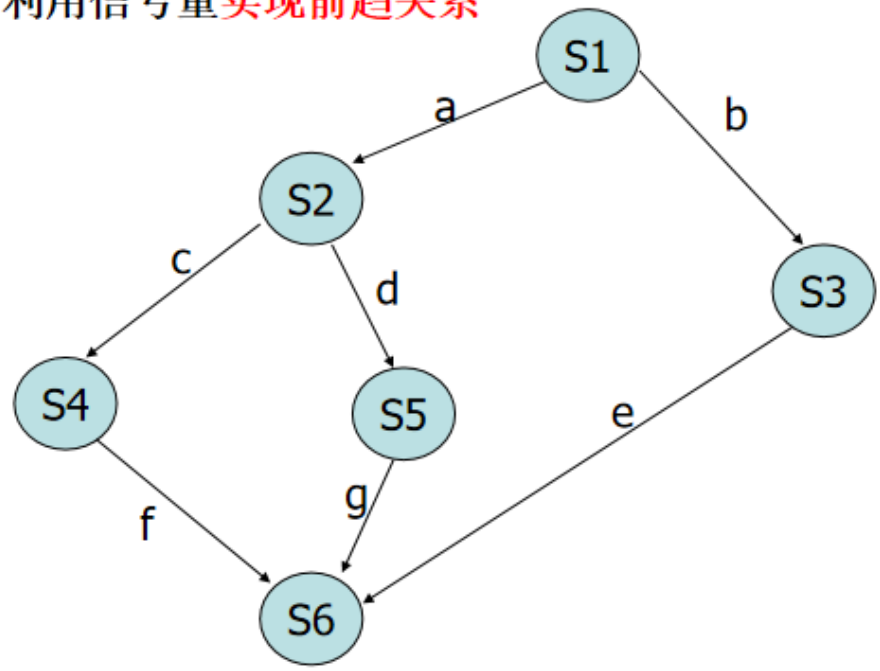
1. 利用信号量实现互斥(2)

```
PB0{
    while(1) {
        wait(mutex);
        critical section
        signal(mutex);
        remainder section
    }
}
```

利用信号量机制实现进程互斥时wait(mutex)和signal(mutex)必须成对地出现。

2.3.3 信号量的应用

2. 利用信号量实现前趋关系



利用信号量来描述前趋关系

```
p1() { S1; signal(a); signal(b);}
p2() { wait(a);S2; signal(c); signal(d);}
p3() { wait(b);S3; signal(e); }
p4() { wait(c);S4; signal(f); }
p5() { wait(d);S5; signal(g); }
p6() { wait(e); wait(f);wait(g);S6; }
main() {
    semaphore  a,b,c,d,e,f,g;
    a.value=b.value=c.value=0;
    d.value=e.value=f.value=0;
    g.value=0;
    cobegin
      p1();p2();p3();p4();p5();p6();
    coend
}
```

生产者消费者问题

2.4.1 生产者—消费者问题

假定有一组生产者和一组消费者进程，通过一个有界缓冲区（容量为 n）发生联系，并进行并发执行。

- 正常情况下，生产者将生产的产品放入缓冲区，消费者从缓冲区取用产品。
- 尽管所有的生产者进程和消费者进程都是以异步方式运行的，但它们之间必须保持同步，即不允许消费者进程到一个空缓冲区去取产品；也不允许生产者进程向一个已装满产品且尚未被取走的缓冲区中投放产品。

（1）用一个数组来表示上述的具有n个缓冲区的缓冲池。

（2）用输入指针in来指示下一个可投放产品的缓冲区，每当生产者进程生产并投放一个产品后，输入指针加1；

（3）用一个输出指针out来指示下一个可从中获取产品的缓冲区，每当消费者进程取走一个产品后，输出指针加1。

（4）缓冲池是组织成循环缓冲的，故应把输入指针加1表示成in=(in+1)% n；输出指针加1表示成out=(out+1) % n。

当(in+1) % n=out时表示缓冲池满；而in=out则表示缓冲池空。

记录型信号量解决



一、利用记录型信号量解决生产者一消费者问题

- 互斥信号量mutex：使诸进程互斥地访问缓冲池（具有n个缓冲区）
- 资源信号量empty、 full：空、满缓冲区数量。
- 又假定这些生产者和消费者相互等效，只要缓冲池未满，生产者便可将消息送入缓冲池；只要缓冲池未空，消费者便可从缓冲池中取走一个消息。
- 对生产者—消费者问题可描述如下：



一、利用记录型信号量解决生产者一消费者问题

对生产者—消费者问题可描述如下：

```
semaphore mutex=1, empty=n, full=0;
item buffer[n];
int in=0, out=0;
```

一、利用记录型信号量解决生产者—消费者问题

```
void producer() {
    do {
        ...
        produce an item in nextp;
        ...
        wait(empty);
        wait(mutex);
        buffer[in]=nextp;
        in=(in+1) % n;
        signal(mutex);
        signal(full);
        }while(TRUE);
}
```

```
void consumer() {
    do {
        wait(full);
        wait(mutex);
        nextc=buffer[out];
        out=(out+1) % n;
        signal(mutex);
        signal(empty);
        consumer the item in nextc;
        ...
        }while(TRUE);
}
```



进程通信

类型

信号量、管道、消息、共享存储器

存储器管理（内存管理）

主要功能

- 存储器管理的主要任务：
  - 一是为多道程序的运行提供良好的环境，方便用户使用存储器；
  - 二是提高存储器的利用率以及从逻辑上扩充内存。

- 存储器管理的功能：
  - 内存分配：分配内存空间、提高存储器的利用率、允许正在运行的程序申请附加的内存空间。
  - 内存保护：每道用户程序都只在自己的内存空间内运行，彼此互不干扰。
  - 地址映射：逻辑地址转换物理地址。
  - 内存扩充：借助于虚拟存储技术，从逻辑上去扩充内存容量。

高速缓存

3．高速缓存

高速缓存是现代计算机结构中的一个重要部件，它是介于寄存器和存储器之间的存储器，主要用于备份主存中较常用的数据，以减少处理机对主存储器的访问次数，这样可大幅度地提高程序执行速度。

高速缓存容量远大于寄存器，而比内存约小两到三个数量级左右，从几十KB到几MB，访问速度快于主存储器。

3．高速缓存

根据程序执行的局部性原理(即程序在执行时将呈现出局部性规律，在一较短的时间内，程序的执行仅局限于某个部分)，将主存中一些经常访问的信息存放在高速缓存中，减少访问主存储器的次数，可大幅度提高程序执行速度。

装入

在多道程序环境下，要使程序运行，必须先为之创建进程。而创建进程的第一件事，便是将程序和数据装入内存。

如何将一个用户源程序变为一个可在内存中执行的程序，通常都要经过以下几个步骤：

编译———链接———装入

将一个用户源程序变为一个可在内存中执行的程序：

首先是编译，由编译程序(Compiler)将用户源代码编译成若干个目标模块(Object Module)；

其次是链接，由链接程序(Linker)将编译后形成的一组目标模块，以及它们所需要的库函数链接在一起，形成一个完整的装入模块(Load Module)；

最后是装入，由装入程序(Loader)将装入模块装入内存。

在将一个装入模块装入内存时，可有如下三种装入方式：

- 1、绝对装入方式
- 2、可重定位装入方式
- 3、动态运行时的装入方式

1、绝对装入方式：

- 编译后，装入前已产生了绝对地址（内存地址），装入时不再作地址重定位。
- 绝对地址的产生：（1）由编译器完成，（2）由程序员编程完成。
- 对（1）而言，编程用符号地址。
- 绝对装入方式只适用于单道程序环境。

程序中所使用的绝对地址，既可在编译或汇编时给出，也可由程序员直接赋予。

但在由程序员直接给出绝对地址时，不仅要求程序员熟悉内存的使用情况，而且一旦程序或数据被修改后，可能要改变程序中的所有地址。

因此，通常是宁可在程序中采用符号地址，然后在编译或汇编时，再将这些符号地址转换为绝对地址。

2. 可重定位装入方式(Relocation Loading Mode)

绝对装入方式只能将目标模块装入到内存中事先指定的位置。在多道程序环境下，编译程序不可能预知所编译的目标模块应放在内存的何处，因此，绝对装入方式只适用于单道程序环境。

2. 可重定位装入方式(Relocation Loading Mode)

在多道程序环境下，所得到的目标模块的起始地址通常是从0开始的，程序中的其它地址也都是相对于起始地址计算的。此时应采用可重定位装入方式，根据内存的当前情况，将装入模块装入到内存的适当位置。

2、可重定位装入

* 通常是把在装入时对目标程序中指令和数据地址的修改过程称为重定位。
* 又因为地址变换通常是在装入时一次完成的，以后不再改变，故称为静态重定位。
* 静态重定位可用于多道程序环境。
* 例：图4—3

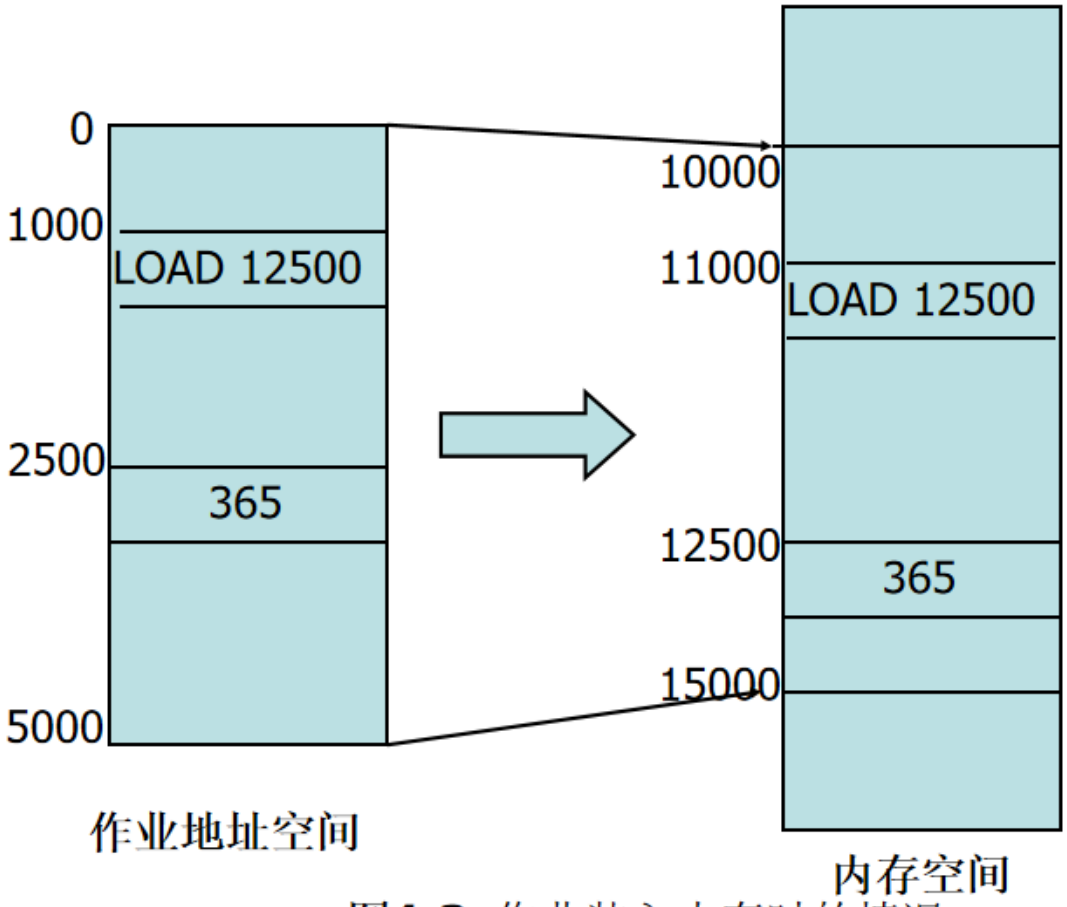


图4-3 作业装入内存时的情况

4.2.1 程序的装入

3. 动态运行时的装入方式(Dynamic Run-time Loading)

可重定位装入方式可将装入模块装入到内存中任何允许的位置，故可用于多道程序环境；但这种方式并不允许程序运行时在内存中移动位置。

程序在内存中的移动，意味着它的物理位置发生了变化，这时必须对程序和数据的地址(绝对地址)进行修改后方能运行。

然而，实际情况是，在运行过程中它在内存中的位置可能经常要改变，此时就应采用动态运行时装入的方式。

- 动态运行时的装入程序在把装入模块装入内存后，并不立即把装入模块中的相对地址转换为绝对地址，而是把这种地址转换推迟到程序真正要执行时才进行相对—绝对地址的转换且有硬件的支持，能保证进程的可移动性。
- 因此，装入内存后的所有地址都仍是相对地址。

链接

根据链接时间的不同，可把链接分成如下三种：

1、静态链接

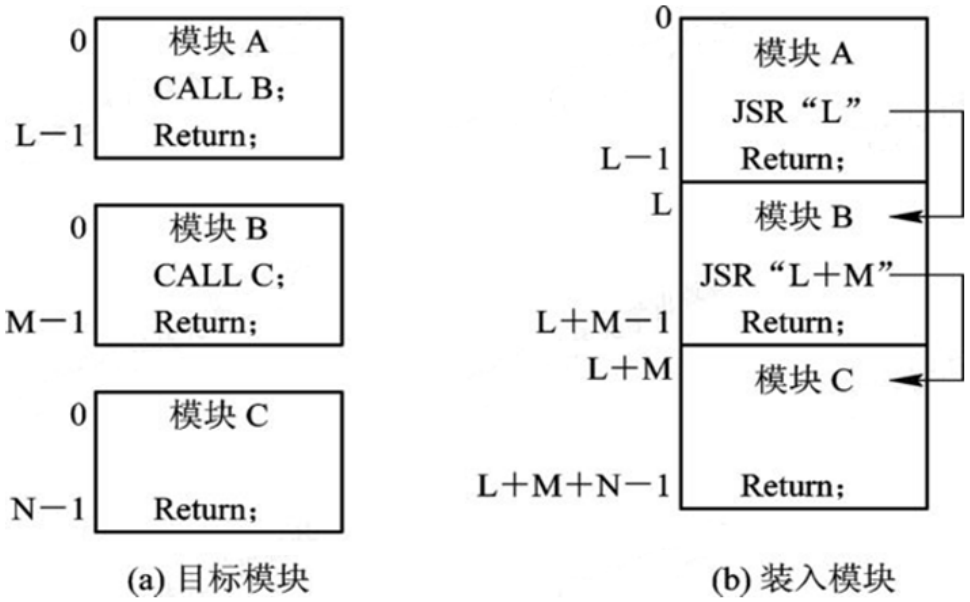
2、装入时动态链接

3、运行时动态链接

1、静态链接

- 在将这几个目标模块装配成一个装入模块时，须解决以下两个问题：
  - a．对相对地址的修改
  - b．变换外部调用符号

1、静态链接



❖ 2、装入时动态链接

* 这是指将用户源程序编译后所得到的一组目标模块，在装入内存时，边装入目标模块边链接。
* 即在装入一个目标模块时，若发生一个外部模块调用事件，将引起装入程序去找出相应的外部目标模块，并将它装入内存，注意修改目标模块中的相对地址。

2. 装入时动态链接

装入时动态链接方式有以下优点：

(1) 便于修改和更新。

(2) 便于实现对目标模块的共享。

3．运行时动态链接(Run-time Dynamic Linking)

在许多情况下，应用程序在运行时，每次要运行的模块可能是不相同的。但由于事先无法知道本次要运行哪些模块，故只能是将所有可能要运行到的模块都全部装入内存，并在装入时全部链接在一起。显然这是低效的，因为往往会有些目标模块根本就不运行。

比较典型的例子是作为错误处理用的目标模块，如果程序在整个运行过程中都不出现错误，则显然就不会用到该模块。

3、运行时动态链接

- 将对某些模块的链接推迟到执行时才执行。
- 即在执行过程中，当发现一个被调用模块尚未装入内存时，立即由OS去找到该模块并将之装入内存，把它链接到调用者模块上。
- 凡在执行过程中未被用到的目标模块，都不会被调入内存和被链接到装入模块上，这样不仅可加快程序的装入过程，而且可节省大量的内存空间。

动态分区

动态分区分配：

- 根据进程的实际需要，动态地为之分配内存空间。
- 在实现可变分区分配时，将涉及到分区分配中所用的数据结构、分区分配算法和分区的分配与回收操作。

一、分区分配中的数据结构

- 为了实现分区分配，系统中必须配置相应的数据结构，用来描述空闲分区和已分配分区的情况，为分配提供依据。常用的数据结构有以下两种形式：
  - 1．空闲分区表：
  - 2．空闲分区链：

❖ 常用的数据结构有以下两种形式：

* 1．空闲分区表：系统中设置一张空闲分区表，用于记录每个空闲分区的情况。每个空闲分区占一个表目，表目中包括分区序号、分区始址及分区的大小等数据项。

❖ 常用的数据结构有以下两种形式：

* 2．空闲分区链：为了实现对空闲分区的分配和链接，在每个分区的起始部分，设置一些用于控制分区分配的信息，以及用于链接各分区所用的前向指针；在分区尾部则设置一后向指针，通过前、后向链接指针，可将所有的空闲分区链接成一个双向链，

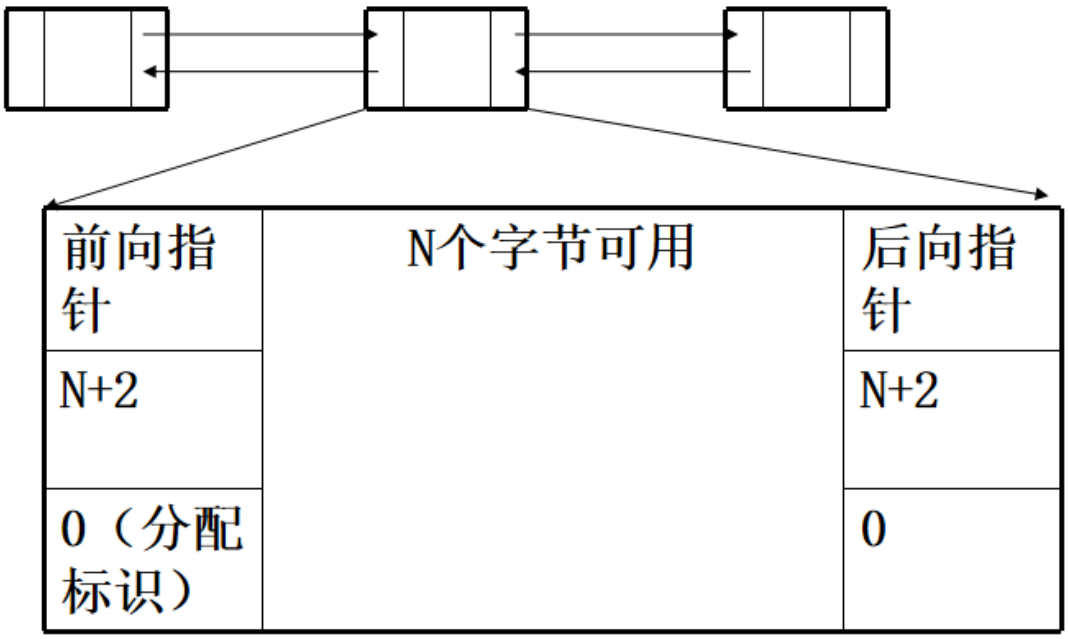


图 4-6 空闲链结构

2．空闲分区链

为了检索方便，在分区尾部重复设置状态位和分区大小表目。当分区被分配出去以后，把状态位由“0”改为“1”，此时，前、后向指针已无意义。

分区分配算法

❖ 二、分区分配算法

❖ 1、基于顺序搜索的动态分区分配算法

* （1）首次适应算法FF。
* （2）循环首次适应算法。
* （3）最佳适应算法
* （4）最坏适应算法

❖ 1、基于顺序搜索的动态分区分配算法

* （1）首次适应算法FF。
  - 要求：分区按低址——高址链接
  - 特点：找到第一个大小满足的分区，划分。有外零头，低址内存使用频繁。每次查找又都是从低址部分开始，会增加查找可用空闲分区时的开销。
* （2）循环首次适应算法。
  - 从1中上次找到的空闲分区的下一个开始查找。
  - 特点：空闲分区分布均匀，提高了查找速度；缺乏大的空闲分区。

❖ 二、分区分配算法

❖1、基于顺序搜索的动态分区分配算法

（3）最佳适应算法(best fit)

所谓“最佳”是指每次为作业分配内存时，总是把能满足要求、又是最小的空闲分区分配给作业，避免“大材小用”。

为了加速寻找，该算法要求将所有的空闲分区按其容量以从小到大的顺序形成一空闲分区链。这样，第一次找到的能满足要求的空闲区，必然是最佳的。

❖二、分区分配算法

❖1、基于顺序搜索的动态分区分配算法

（3）最佳适应算法(best fit)

孤立地看，最佳适应算法似乎是最佳的，然而在宏观上却不一定。因为每次分配后所切割下来的剩余部分总是最小的，这样，在存储器中会留下许多难以利用的小空闲区。

❖1、基于顺序搜索的动态分区分配算法

（4）最坏适应算法(worst fit)

该算法要求将所有的空闲分区按其容量以从大到小的顺序形成一空闲分区链，查找时只要看第一个分区能否满足作业要求。

其优点是可使剩下的空闲区不至于太小，产生碎片的几率最小，对中、小作业有利，同时最坏适应分配算法查找效率很高。

但是其缺点也是明显的，它会使存储器中缺乏大的空闲分区。

（4）最坏适应算法(worst fit)

最坏适应算法与前面所述的首次适应算法、循环首次适应算法、最佳适应算法一起，也称为顺序搜索法。

❖2、基于索引搜索的动态分区分配算法

（1）快速适应算法(quick fit)(又称为分类搜索法)

该算法是将空闲分区根据其容量大小进行分类，对于每一类具有相同容量的所有空闲分区，单独设立一个空闲分区链表，这样，系统中存在多个空闲分区链表，同时在内存中设立一张管理索引表，该表的每一个表项对应了一种空闲分区类型，并记录了该类型空闲分区链表表头的指针。

（1）快速适应算法

空闲分区的分类是根据进程常用的空间大小进行划分，如2 KB、4 KB、8 KB等，对于其它大小的分区，如7 KB这样的空闲区，既可以放在8 KB的链表中，也可以放在一个特殊的空闲区链表中。

（1）快速适应算法

该算法的优点是:

（1）查找效率高，仅需要根据进程的长度，寻找到能容纳它的最小空闲区链表，并取下第一块进行分配即可。

（2）该算法在进行空闲分区分配时，不会对任何分区产生分割，所以能保留大的分区，满足对大空间的需求，也不会产生内存碎片。

（1）快速适应算法

该算法的缺点：

（1）在分区归还主存时算法复杂，系统开销较大。

（2）此外，该算法在分配空闲分区时是以进程为单位，一个分区只属于一个进程，因此在为进程所分配的一个分区中，或多或少地存在一定的浪费。

（3）空闲分区划分越细，浪费则越严重，整体上会造成可观的存储空间浪费，这是典型的以空间换时间的作法。

❖ 三、分区分配操作

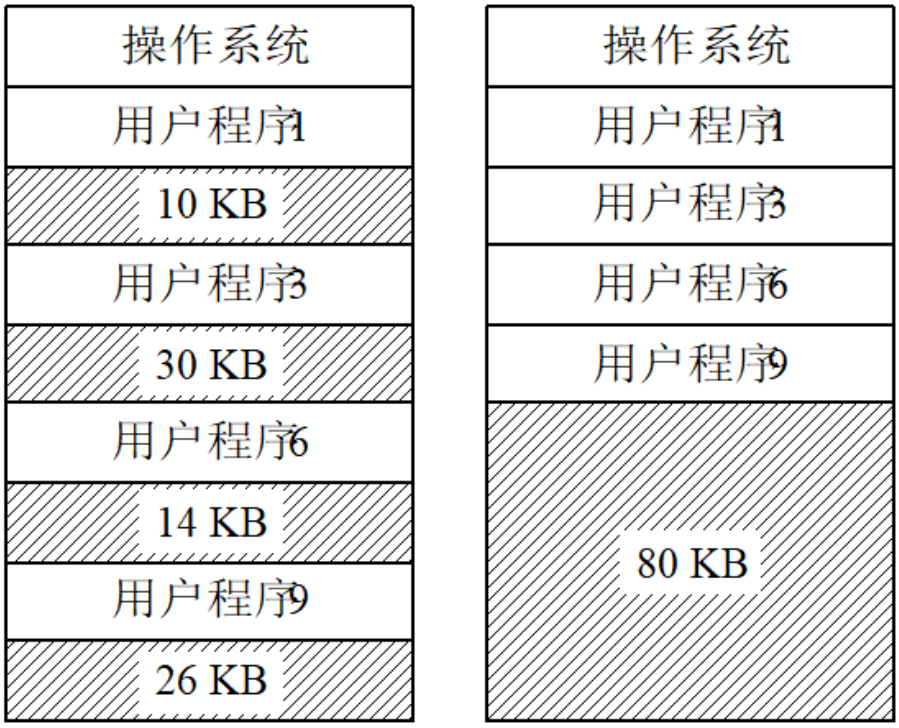
* 2. 回收内存：
* 当进程运行完毕释放内存时，系统根据回收区的首址，从空闲区链(表)中找到相应的插入点，此时可能出现以下四种情况之一：
  - （1）回收区上邻空闲区：合并，改大小
  - （2）回收区下邻空闲区：合并，改大小，首址。
  - （3）回收区同时上、下邻空闲区：合并，改大小。
  - （4）不邻接，则建立一新表项。

动态可重定位分区分配（零散内存的紧凑拼接）

❖ 1. 动态重定位的引入

* 在连续分配方式中，必须把一个系统或用户程序装入一连续的内存空间。如果在系统中只有若干个小的分区，即使它们容量的总和大于要装入的程序，但由于这些分区不相邻接，也无法把该程序装入内存。

1. 动态重定位的引入



(a) 紧凑前　　(b) 紧凑后

图 4-9　紧凑的示意

1．动态重定位的引入

在图4-9(a)中，如果现在有一作业到达，要求获得40 KB的内存空间，由于必须为它分配一连续空间，故此作业无法装入。

这种不能被利用的小分区称为“零头”或“碎片”。

为了解决以上问题就可以通过“拼接”或“紧凑”方法来移动作业，从而拼接成一个大分区以装入大作业。

❖ 1. 动态重定位的引入

* 拼接或紧凑
  - 通过移动内存中作业的位置，以把原来多个分散的小分区拼接成一个大分区。
  - 需对移动了的程序或数据进行重定位，是动态（因作业已经装入）

- 原理：内存紧凑
- 地址映射
  - 地址空间：
    - 一个应用程序(源程序)经编译后，通常会形成若干个目标程序，这些目标程序的地址都是从“0”开始的，程序中的其它地址都是相对于起始地址计算的。
    - 由这些地址所形成的地址范围称为地址空间，其中的地址称为逻辑地址或相对地址。

- 地址映射
  - 内存空间：
    - 由内存中的一系列单元所限定的地址范围称为内存空间，其中的地址称为物理地址。

  - 重定位：把作业地址空间中使用的逻辑地址变换成主存中的物理地址的过程。

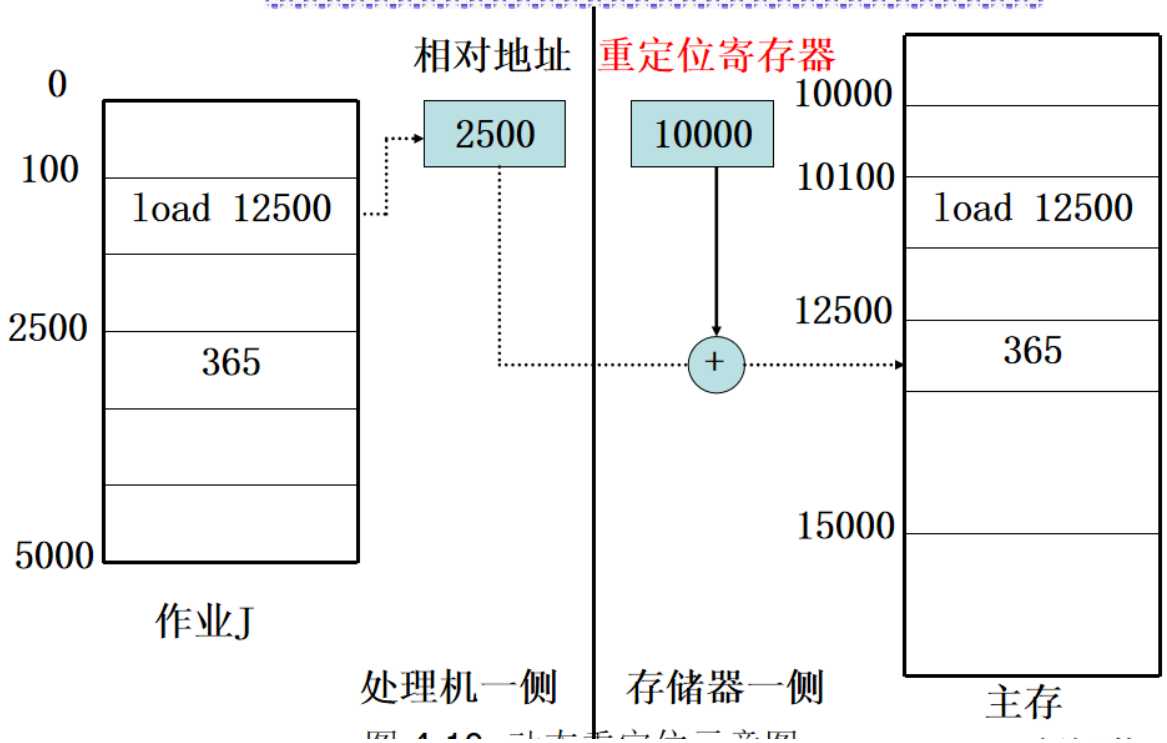
2．动态重定位的实现

在动态运行时装入的方式中，作业装入内存后的所有地址都仍然是相对地址，将相对地址转换为物理地址的工作，被推迟到程序指令要真正执行时进行。

2．动态重定位的实现

为使地址的转换不会影响到指令的执行速度，必须有硬件地址变换机构的支持，即须在系统中增设一个重定位寄存器，用它来存放程序(数据)在内存中的起始地址。

程序在执行时，真正访问的内存地址是相对地址与重定位寄存器中的地址相加而形成的。图4-10示出了动态重定位的实现原理。



2．动态重定位的实现

地址变换过程是在程序执行期间，随着对每条指令或数据的访问自动进行的，故称为动态重定位。

当系统对内存进行了“紧凑”而使若干程序从内存的某处移至另一处时，不需对程序做任何修改，只要用该程序在内存的新起始地址，去置换原来的起始地址即可。

3．动态重定位分区分配算法

动态重定位分区分配算法与动态分区分配算法基本上相同，差别仅在于：在这种分配算法中，增加了紧凑的功能，通常，在找不到足够大的空闲分区来满足用户需求时进行紧凑。图4-11示出了动态重定位分区分配算法。

3、动态重定位分区分配算法

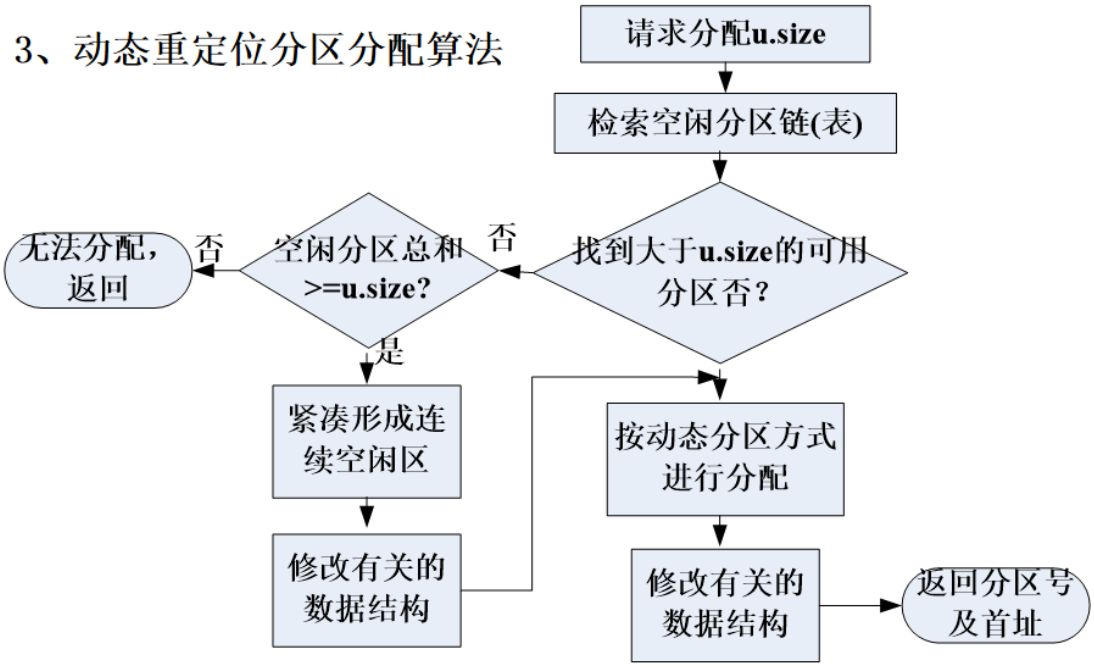


图 4-11 动态重定位分区分配算法流程图

对换

4.3.5 对换

1．对换(Swapping)的引入

在多道程序环境下，一方面，在内存中的某些进程由于某事件尚未发生而被阻塞运行，但它却占用了大量的内存空间，甚至有时可能出现在内存中所有进程都被阻塞而迫使CPU停止下来等待的情况；

另一方面，却又有着许多作业在外存上等待，因无内存而不能进入内存运行的情况。

显然这对系统资源是一种严重的浪费，且使系统吞吐量下降。为了解决这一问题，在系统中又增设了对换(也称交换)设施。

❖ 1、 对换的引入

* 对换是指把内存中暂时不能运行的进程或者暂时不用的程序和数据，调出到外存上，以便腾出足够的内存空间。再把已具备运行条件的进程或进程所需要的程序和数据，调入内存。
* 对换是提高内存利用率的有效措施。

❖ 2、 对换类型：

- 整体对换：进程对换，解决内存紧张，可进一步提高内存的利用率，广泛地应用于分时系统中。
- 部分对换：页面对换/分段对换，其目的是为了支持虚拟存储系统。

* 为了实现进程对换，系统必须能实现三方面的功能：对换空间的管理、进程的换出，以及进程的换入。

❖ 3、 对换空间的管理

* 在具有对换功能的OS中，通常把外存分为文件区和对换区。前者用于存放文件，后者用于存放从内存换出的进程。
* 对文件区管理的主要目标，是提高文件存储空间的利用率，为此，对文件区采取离散分配方式。

❖ 3、 对换空间的管理

* 进程在对换区中驻留的时间是短暂的，对换操作又较频繁，故对对换空间管理的主要目标，是提高进程换入和换出的速度。
* 为此，采取的是连续分配方式，较少考虑外存中的碎片问题。
* 采用数据结构和分配回收类似于动态分区分配。

❖ 3、 对换空间的管理

* 为了能对对换区中的空闲盘块进行管理，在系统中应配置相应的数据结构，以记录外存的使用情况。
* 其形式与内存在动态分区分配方式中所用数据结构相似，即同样可以用空闲分区表或空闲分区链。
* 在空闲分区表中的每个表目中应包含两项， 即对换区的首址及其大小，它们的单位是盘块号和盘块数。

- 4、 换出与换入
  - （1）、换出
    - a. 选出被换出进程：
      因素：优先级，驻留时间，进程状态
    - b. 换出过程：
      - 首先选择处于阻塞状态且优先级最低的进程作为换出进程。
      - 然后启动盘块，将该进程的程序和数据传送到磁盘的对换区上。
      - 对于共享段：计数减1，是0则换出，否则不换
      - 修改PCB和MCB（或内存分配表）

- 3 换出与换入
  - （2）、换入：
    - a. 选择换入进程：系统应定时地查看所有进程的状态，从中找出“就绪”状态但已换出的进程，将其中换出时间（换出到磁盘上）最久的进程作为换入进程。
    - b. 申请内存。
    - c. 换入：
    - 直至已无可换入的进程或无可换出的进程为止。

分页存储管理（跟紧凑一样：为了解决碎片问题）

- 连续分配引起:碎片
- 碎片问题的解决：紧凑方式消耗系统开销。
- 离散分配
  - 分页、分段、段页

- 1. 页面

1）页面和物理块：逻辑空间和内存空间

- 将一个进程的逻辑地址空间分成若干个大小相等的片，称为页面或页，并为各页加以编号，从0开始，如第0页、第1页等。
- 相应地，也把内存空间分成与页面相同大小的若干个存储块，称为(物理)块，也同样为它们加以编号，如0＃块、1＃块等等。

❖ 1. 页面

1) 页面和物理块：逻辑空间和内存空间

* 在为进程分配内存时，以块为单位将进程中的若干个页分别装入到多个可以不相邻接的物理块中。
* 由于进程的最后一页经常装不满一块而形成了不可利用的碎片，称之为“页内碎片”。

❖ 1. 页面

* 2) 页面大小
  - 页太小：减少了内存碎片的总空间，有利于提高内存利用率；但会使每个进程占用较多的页面，从而导致进程的页表过长，占用大量内存；此外，还会降低页面换进换出的效率。
  - 页太大：可以减少页表的长度，提高页面换进换出的速度，但却又会使页内碎片增大。
  - 页面大小应是2的幂，通常为1KB～8KB。

❖ 2. 地址结构

* 31 12 11 0

页号P	位移量W

* 其中0～11位为位移量W(或称为页内地址d)，即每页的大小为4 KB；
* 12～31位为页号，地址空间最多允许有1 M页。

❖ 2. 地址结构

* 对于某特定机器，其地址结构是一定的。若给定一个逻辑地址A；页大小L；则页号P和页内地址d可按下式求得：

$$P = INT\left[\frac{A}{L}\right]$$

$$d = [A] \ MOD \ L$$

如： A＝2170 B，L为1 KB. 则P=2, d=122

页表

页面映射表，简称页表。

- 每个进程有一张页表。
- 在进程地址空间内的所有页，依次在页表中有一页表项，其中记录了相应页在内存中对应的物理块号。

3. 页表

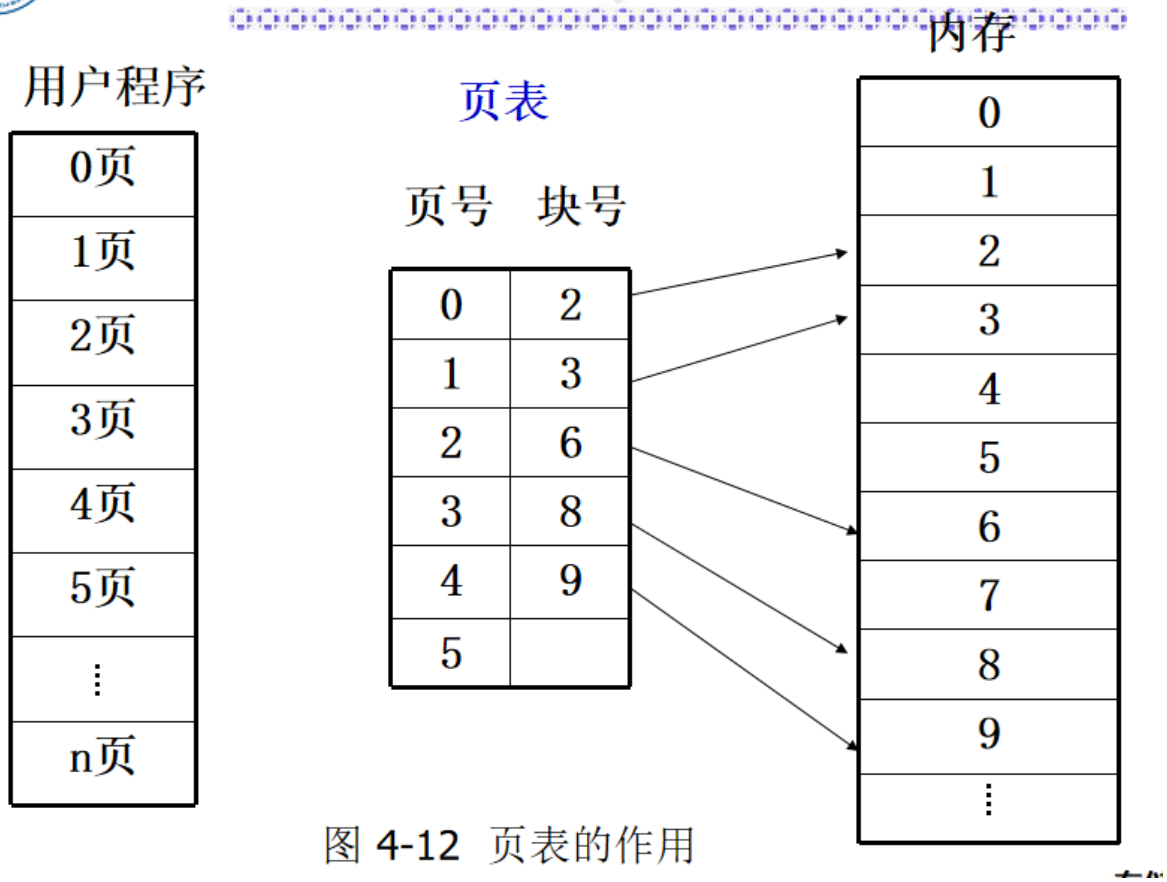


图 4-12 页表的作用

- 在配置了页表后，进程执行时，通过查找该表，即可找到每页在内存中的物理块号。
- 页表的作用是实现从页号到物理块号的地址映射。
  - 地址变换机构的基本任务是：实现从逻辑地址到物理地址的转换，即页号到块号的转变。
  - 借助页表来完成地址变换。

❖ 一、基本地址变换机构：

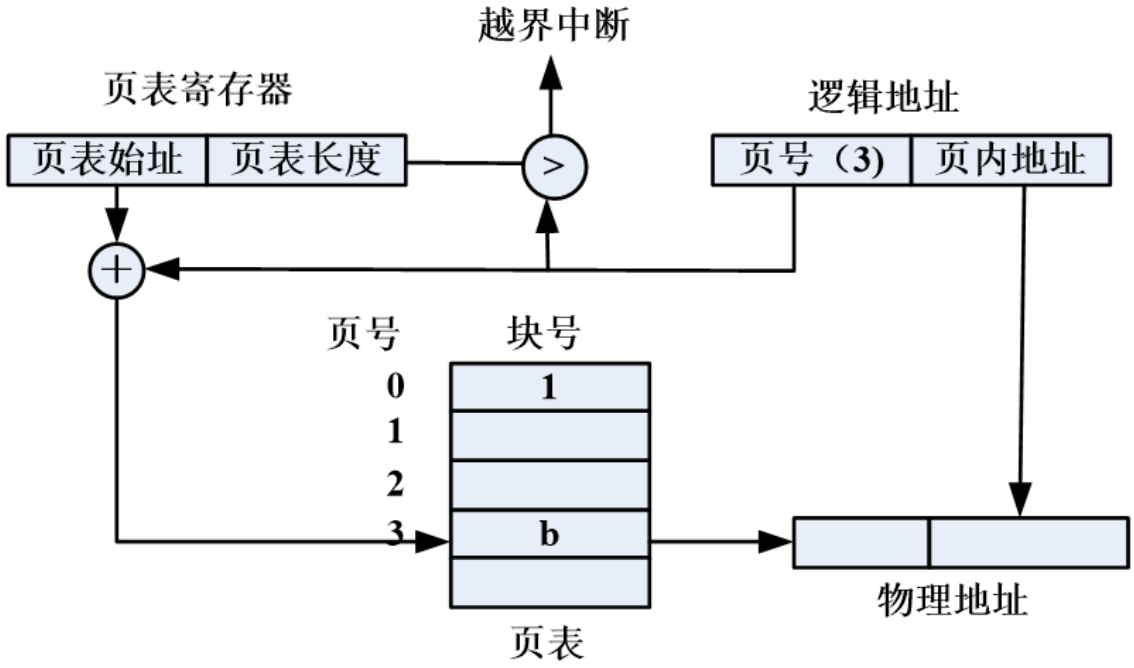
- 页表的功能可以由一组专门的寄存器来实现。一个页表项用一个寄存器。
- 这样有利于提高地址变换的速度；
- 但由于寄存器成本较高，且大多数现代计算机的页表又可能很大，使页表项的总数可达几千甚至几十万个，显然这些页表项不可能都用寄存器来实现。
- 因此，页表大多驻留在内存中。

❖ 一、基本地址变换机构：

- 在系统中只设置一个页表寄存器PTR(Page-Table Register)，在其中存放页表在内存的始址和页表的长度。

- 每个进程对应一页表，其信息（如长度、始址）放在PCB中，执行时将其首地址装入页表寄存器。

1. 基本的地址变换机构



分页系统地址变换机构

快表

- 不具快表，则需两次访问内存。
  - （1）访页表，以形成物理地址
  - （2）从所得的物理地址中获得所需数据(或向此地址中写入数据)
- 因此，采用这种方式将使计算机的处理速度降低近1/2。

为了提高地址变换速度，可在地址变换机构中增设一个具有并行查寻能力的特殊高速缓冲寄存器，又称为“联想寄存器”，或称为“快表”，用以存放当前访问的那些页表项。

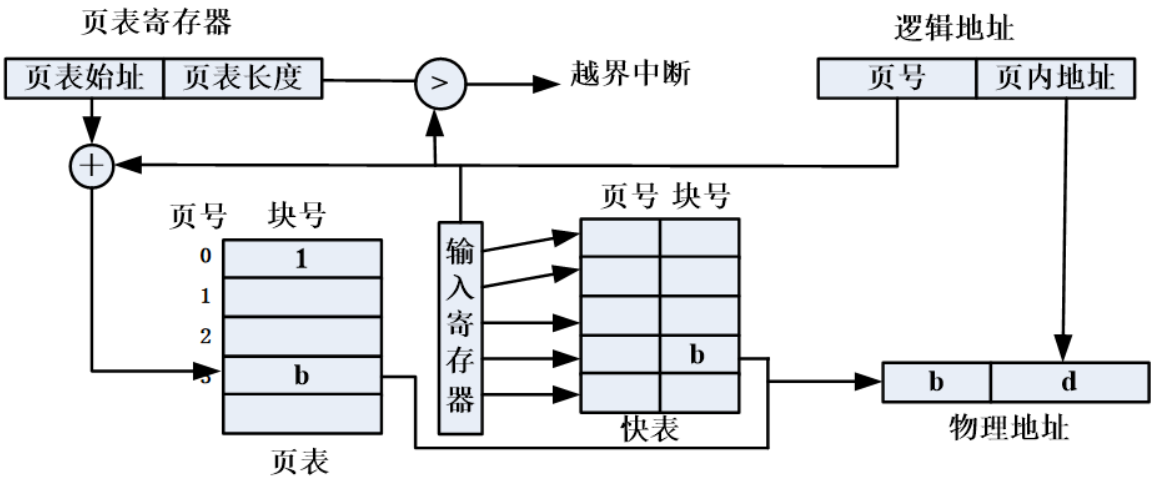
具有快表的地址变换过程是：

如果可直接从快表中读出某页所对应的物理块号，直接进行地址变换。

否则，则还须再访问内存中的页表，同时，重新修改快表。

但如果联想寄存器已满，则OS必须找到一个老的且已被认为不再需要的页表项，将它换出。

图 4-14示出了具有快表的地址变换机构。



具有快表的地址变换机构

一些概念（内存有效访问时间，命中率）

❖ 内存的有效访问时间：

- 从进程发出指定逻辑地址的访问请求，经过地址变换，到在内存中找到对应的实际物理地址单元并取出数据，所需要花费的总时间，称为内存的有效访问时间（EAT:Effective Access Time）。

❖ 命中率：

- 是指使用快表并在其中成功查找到所需页面的表项的比率。

❖ 假设访问一次内存的时间为t：

❖ 在基本分页存储管理方式中，

- EAT = t + t = 2t。

❖ 在引入了快表的分页存储管理方式中，

$$EAT = a \times \lambda + (t + \lambda)(1 - a) + t = 2t + \lambda - t \times a$$

- 其中，λ 表示查找快表所需要的时间，a表示命中率。

优缺点



分页存储管理方案的评价

❖ 优点

- 有效解决存储器的零头问题，能在更高的程度上进行多道程序设计，从而相应提高了存储器和CPU的利用率。

❖ 缺点

- 采用动态地址变换增加计算机成本和降低CPU的速度。
- 表格占内存空间，费时来管理表格。
- 存在页内碎片
- 作业动态的地址空间受内存容量限制。

分段存储管理

4.5 分段存储管理

❖ 4.5.1 引入

- 引入分段存储管理方式， 主要是为了满足用户和程序员的下述一系列需要：
  - （1）方便编程；
  - （2）信息共享；
  - （3）信息保护；
  - （4）动态增长；（如数据段的增长）
  - （5）动态链接；

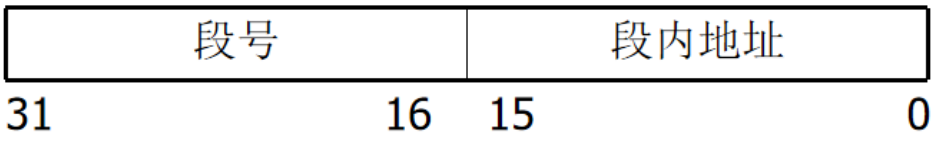
❖ 基本思想

- 作业的地址空间被划分为若干个段，每个段定义了一组逻辑信息，每段有自己的名字。通常可用一个段号来代替段名。
- 每个段都从0开始编址，并采用一段连续的地址空间。段的长度由相应的逻辑信息组的长度决定，因而各段长度不等。
- 段式管理程序以段为单位分配内存，然后通过地址映射机构将进程的逻辑地址转换成实际的内存物理地址。

❖ 1. 实现原理

- 对用户而言，分段是2维的，段长不固定，每个段定义一组逻辑上完整的程序或数据。
- 逻辑地址：段号+段内地址。

❖ 分段地址中的地址具有如下结构：



❖ 段表：

- 逻辑段—map—物理段

段表

◆ 2. 段表

动态分区分配方式中，系统为整个进程分配一个连续的内存空间。而在分段式存储管理系统中，则是为每个分段分配一个连续的分区，而进程中的各个段可以离散地移入内存中不同的分区中。

◆ 2. 段表

为使程序能正常运行，为每个进程建立一张段映射表，简称“段表”。每个段在表中占有一个表项，其中记录了该段在内存中的起始地址(又称为“基址”)和段的长度，如图4-17 所示。

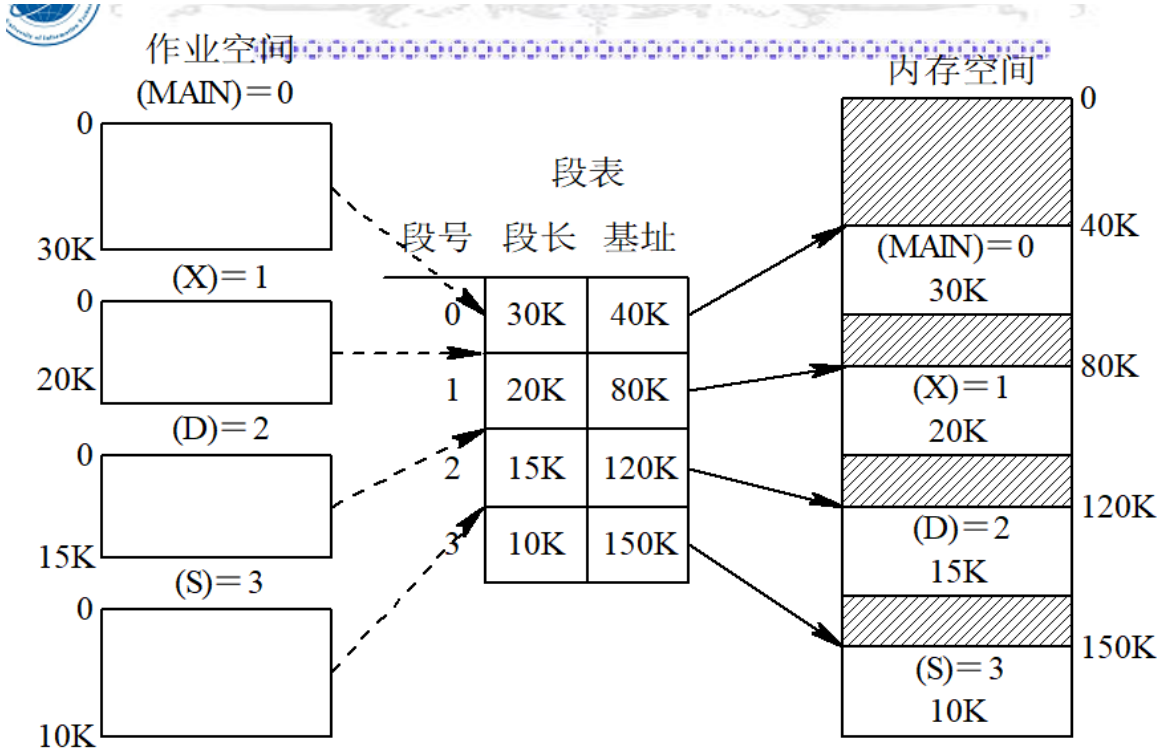


图 4-17 利用段表实现地址映射

段表可以存放在一组寄存器中，这样有利于提高地址转换速度，但更常见的是将段表放在内存中。

3. 地址变换机构

段表寄存器：用于存放段表始址和段表长度TL。

在进行地址变换时，系统将逻辑地址中的段号S与段表长度TL进行比较：

若S>TL，表示段号太大，是访问越界，于是产生越界中断信号；

3．地址变换机构

若未越界，则根据段表的始址和该段的段号，计算出该段对应段表项的位置，从中读出该段在内存的起始地址，然后，再检查段内地址d是否超过该段的段长SL。若超过，即d>SL，同样发出越界中断信号；若未越界，则将该段的基址d与段内地址相加，即可得到要访问的内存物理地址。

图4-18示出了分段系统的地址变换过程。

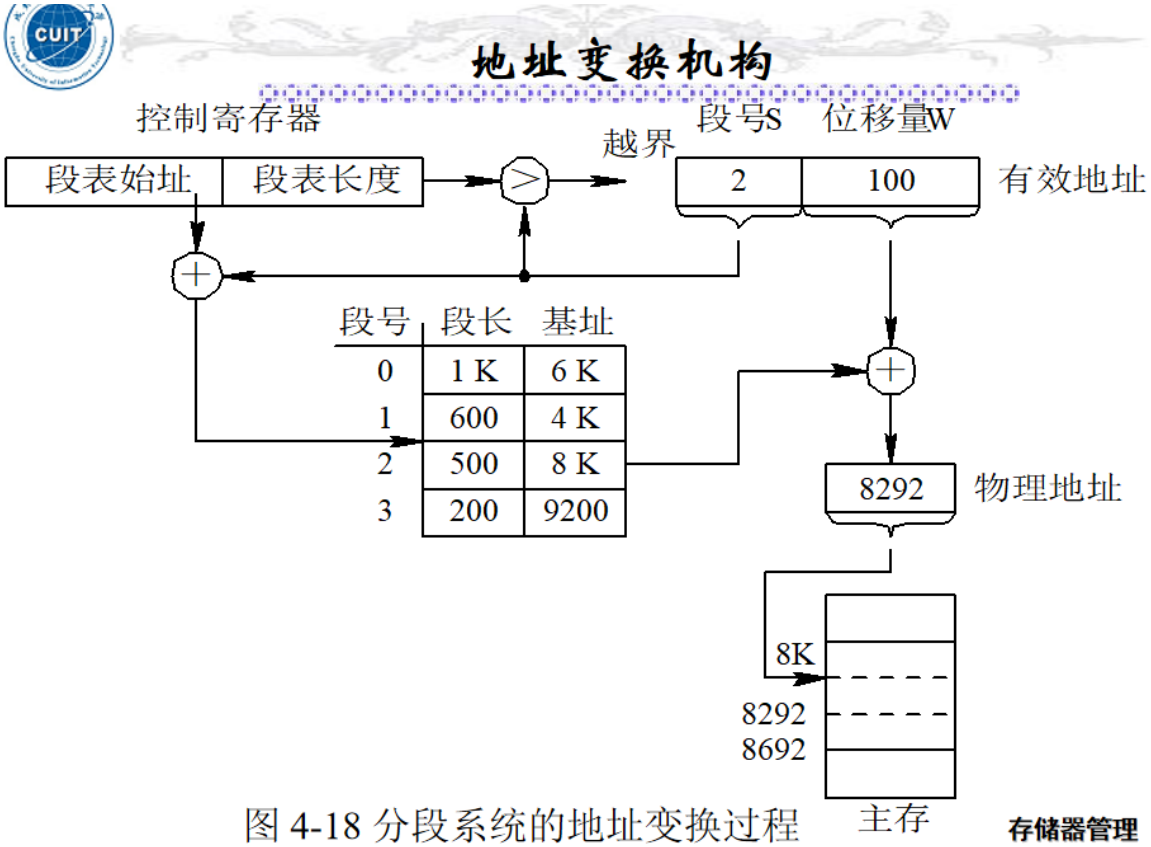


图 4-18 分段系统的地址变换过程

- 对内存两次以上访问，可采用高速联想寄存器，加快查找速度。

对比分页和分段

4. 分页与分段的主要区别：

- 分页和分段系统有许多相似之处：如都采用离散分配方式，且都要通过地址映射机构来实现地址变换。
- 但在概念上两者完全不同，主要表现在下述三个方面。
  - （1）页是信息的物理单位，段是逻辑单位。
  - （2）页的大小固定，且由系统决定。而段的长度不固定（由用户指定）
  - （3）分页的用户程序地址空间是一维的，而分段系统中，用户程序的地址空间是二维的。

优缺点



4.5 分段存储管理

- 性能评价
  - 优点
    - 便于程序模块化处理和处理变化的数据结构；
    - 便于共享分段；
    - 便于动态链接。
  - 缺点
    - 地址变换费时，管理表格，硬件支持，使OS复杂；
    - 为满足段的动态增长和减少碎片，要用拼接技术；
    - 段长不定，管理困难；段长受内存可用区的限制。

段页式存储管理（未完）

虚拟存储器管理（多了个虚拟）

前面所介绍的各种存储器管理方式有一个共同的特点，即它们都要求将一个作业全部装入内存后方能运行，于是，出现了下面这样两种情况：

(1) 有的作业很大，其所要求的内存空间超过了内存总容量，作业不能全部被装入内存，致使该作业无法运行。

(2) 有大量作业要求运行，但由于内存容量不足以容纳所有这些作业，只能将少数作业装入内存让它们先运行，而将其它大量的作业留在外存上等待。

❖ 1. 引入

* (1) 常规存储管理的特征：
  - 一次性（指全部装入）、
  - 驻留性（指驻留在内存不换出，直至作业运行结束）

* 一次性及驻留性使许多在程序运行中不用或暂不用的程序(数据)占据了大量的内存空间，使得一些需要运行的作业无法装入运行。

* 现在要研究的问题是：一次性及驻留性在程序运行时是否是必需的。

(2) 局部性原理

早在1968年，Denning.P就曾指出：程序在执行时将呈现出局部性规律：

1) 程序执行时，除了少部分的转移和过程调用指令外，在大多数情况下仍是顺序执行的。

2) 过程调用将会使程序的执行轨迹由一部分区域转至另一部分区域，但经研究看出，过程调用的深度在大多数情况下都不超过5。

（2）局部性原理

3）程序中存在许多循环结构，这些虽然只由少数指令构成，但是它们将多次执行。

4）程序中还包括许多对数据结构的处理，如对数组进行操作，它们往往都局限于很小的范围内。

* （2）局部性原理
  - 即在一较短的时间内，程序的执行仅局限于某个部分；相应地，它所访问的存储空间也局限于某个区域。
  - 时间局部性：如循环执行
  - 空间局部性：如顺序执行。

虚拟存储器

* （3）虚拟存储器
  - 具有请求调入功能和置换功能，能从逻辑上对内存容量进行扩充的一种存储器系统。
  - 其逻辑容量由内存容量和外存容量之和所决定，其运行速度接近于内存速度，而每位的成本却又接近于外存。
  - 实质：以时间换空间，但时间牺牲不大。
  - 虚拟存储技术是一种性能非常优越的存储器管理技术，故被广泛地应用于大、中、小型机器和微型机中。

❖ 需要动态重定位，建立在离散分配的存储管理方式的基础上，采用下述方式之一实现。

* （1）请求分页系统

* （2）请求分段系统

请求分页系统

（1）请求分页系统

在分页系统的基础上，增加了请求调页功能和页面置换功能所形成的页式虚拟存储系统。它允许只装入少数页面的程序(及数据)，便启动运行。

以后，再通过调页功能及页面置换功能，陆续地把即将要运行的页面调入内存，同时把暂不运行的页面换出到外存上。

（1）请求分页系统

置换时以页面为单位。为了能实现请求调页和置换功能，系统必须提供必要的硬件支持和相应的软件。

（1）请求分页系统—以页为单位转换

1）硬件支持。

① 请求分页的页表机制，它是在纯分页的页表机制上增加若干项而形成的，作为请求分页的数据结构；

② 缺页中断机构，即每当用户程序要访问的页面尚未调入内存时，便产生一缺页中断，以请求OS将所缺的页调入内存；

③ 地址变换机构，它同样是在纯分页地址变换机构的基础上发展形成的。

（1）请求分页系统—以页为单位转换

2）实现请求分页的软件

实现请求调页的软件和实现页面置换的软件。

请求分段系统

（2）请求分段系统

在分段系统的基础上，增加了请求调段及分段置换功能后所形成的段式虚拟存储系统。它允许只装入少数段(而非所有的段)的用户程序和数据，即可启动运行。以后再通过调段功能和段的置换功能将暂不运行的段调出，同时调入即将运行的段。

- （2）请求分段系统
  - 置换是以段为单位进行的。
  - 硬件支持：
    1）请求分段的段表结构
    2）缺段中断
    3）地址变换机构
  - 需实现请求分段机制的软件（置换软件等）

- （1）多次性：局部装入，多次装入。
- （2）对换性：允许在作业运行过程中进行换进、换出
- （3）虚拟性：能从逻辑上扩充内存容量。

- 虚拟性是以多次性和对换性为基础的，而多次性和对换性又必须建立在离散分配的基础上。

- 在分页系统的基础上，增加了请求调页功能和页面置换功能。
- 每次调入和换出的基本单位都是长度固定的页面。

请求分页存储管理方式

- 在分页系统的基础上，增加了请求调页功能和页面置换功能。
- 每次调入和换出的基本单位都是长度固定的页面。

❖ 1. 请求分页中的硬件支持

* （1）页表机制

页表项：

页号	物理块号	状态位P	访问字段A	修改位M	外存地址

（2）缺页中断机构

在请求分页系统中，每当所要访问的页面不在内存时，便产生一缺页中断，请求OS将所缺之页调入内存。

缺页中断作为中断，它们同样需要经历诸如保护CPU环境、分析中断原因、转入缺页中断处理程序进行处理、恢复CPU环境等几个步骤。

（2）缺页中断机构

但缺页中断又是一种特殊的中断，它与一般的中断相比，有着明显的区别，主要表现在下面两个方面：

1）在指令执行期间产生和处理中断信号。

通常，CPU都是在一条指令执行完后，才检查是否有中断请求到达。若有，便去响应，否则，继续执行下一条指令。

然而，缺页中断是在指令执行期间，发现所要访问的指令或数据不在内存时所产生和处理的。

2）一条指令在执行期间，可能产生多次缺页中断。

（2） 缺页中断机构

可在指令执行期间产生，转入缺页中断处理程序。

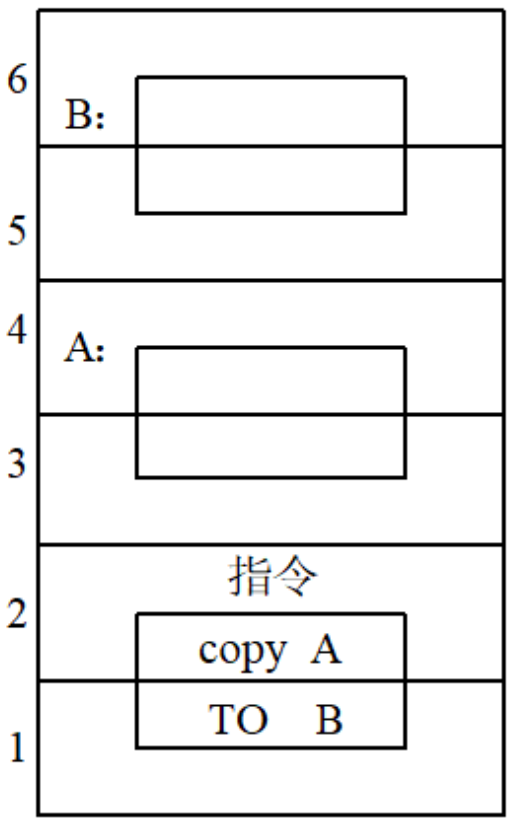


图 4-24 涉及6次缺页中断的指令

（3）地址变换机构

在分页系统地址变换机构的基础上，增加了某些功能如产生和处理缺页中断，以及从内存中换出一页的功能等等。

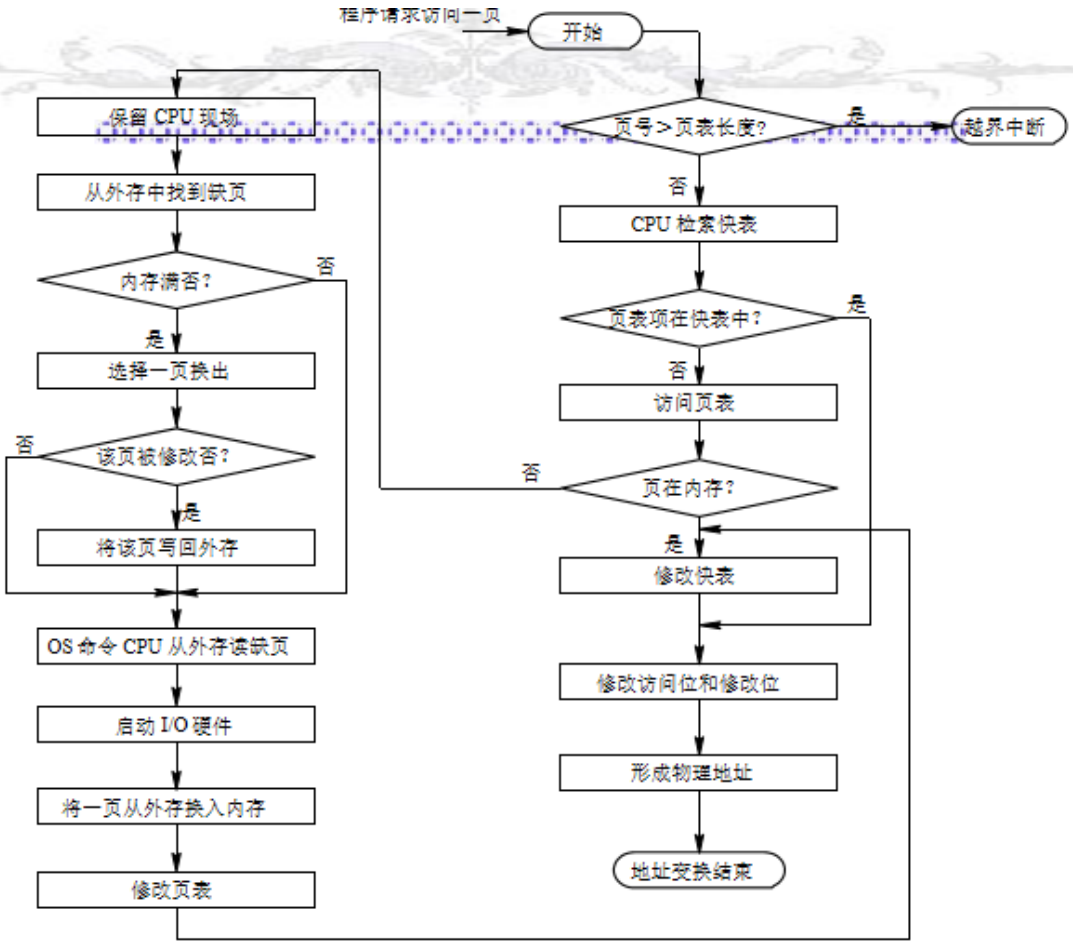


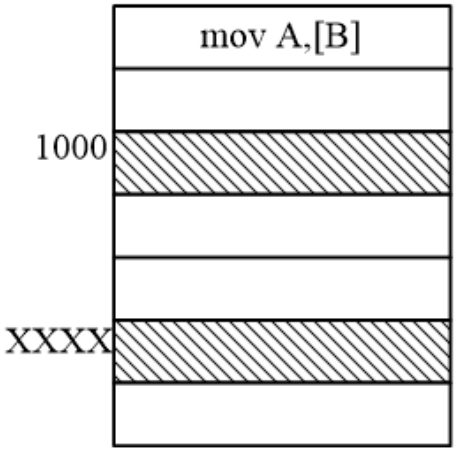
图 4-25　请求分页中的地址变换过程

内存分配

2. 内存分配策略和分配算法

❖ （1）最小物理块数

- 是指能保证进程正常运行所需的最小物理块数。
- 不同的作业要求不同。
- 进程应获得的最少物理块数与计算机的硬件结构有关，取决于指令的格式、功能和寻址方式。



例如：

- 若是单地址指令且采用直接寻址方式，则所需的最少物理块数为2。其中，一块是用于存放指令的页面，另一块则是用于存放数据的页面。
- 如果该机器允许间接寻址时，则至少要求有三个物理块。
- 对于某些功能较强的机器，其指令长度可能是两个或多于两个字节，因而其指令本身有可能跨两个页面，且源地址和目标地址所涉及的区域也都可能跨两个页面。

（2） 内存分配策略

在请求分页系统中，可采取两种内存分配策略，即固定和可变分配策略。在进行置换时，也可采取两种策略，即全局置换和局部置换。于是可组合出以下三种适用的策略。

1)．固定分配局部置换。

2)．可变分配全局置换

3)．可变分配局部置换

（2） 内存分配策略

在请求分页系统中，可采取两种内存分配策略，即固定和可变分配策略。在进行置换时，也可采取两种策略，即全局置换和局部置换。于是可组合出以下三种适用的策略。

1)．固定分配局部置换。

2)．可变分配全局置换

3)．可变分配局部置换

（2） 内存分配策略—物理块的分配

1）固定分配局部置换

这是指基于进程的类型(交互型或批处理型等)，或根据程序员、程序管理员的建议，为每个进程分配一定数目的物理块，在整个运行期间都不再改变。

采用该策略时，如果进程在运行中发现缺页，则只能从该进程在内存的n个页面中选出一个页换出，然后再调入一页，以保证分配给该进程的内存空间不变。

1）固定分配局部置换

实现这种策略的困难在于：应为每个进程分配多少个物理块难以确定。

若太少，会频繁地出现缺页中断，降低了系统的吞吐量；

若太多，又必然使内存中驻留的进程数目减少，进而可能造成CPU空闲或其它资源空闲的情况，而且在实现进程对换时，会花费更多的时间。

2）可变分配全局置换

最易于实现，已用于若干个OS中。

在采用这种策略时，先为系统中的每个进程分配一定数目的物理块，而OS自身也保持一个空闲物理块队列。

当某进程发现缺页时，由系统从空闲物理块队列中取出一个物理块分配给该进程，并将欲调入的(缺)页装入其中。

这样，凡产生缺页(中断)的进程，都将获得新的物理块。

仅当空闲物理块队列中的物理块用完时，OS才能从内存中选择一页调出，该页可能是系统中任一进程的页，这样，自然又会使那个进程的物理块减少，进而使其缺页率增加。

3）可变分配局部置换

这同样是基于进程的类型或根据程序员的要求，为每个进程分配一定数目的物理块，但当某进程发现缺页时，只允许从该进程在内存的页面中选出一页换出，这样就不会影响其它进程的运行。

如果进程在运行中频繁地发生缺页中断，则系统须再为该进程分配若干附加的物理块，直至该进程的缺页率减少到适当程度为止；

反之，若一个进程在运行过程中的缺页率特别低，则此时可适当减少分配给该进程的物理块数，但不应引起其缺页率的明显增加。

（3）物理块分配算法

❖ 1)、平均分配算法：

- 将系统中所有可供分配的物理块，平均分配给各个进程。
- 这种方式貌似公平，但实际上是不公平的，因为它未考虑到各进程本身的大小。

（3）物理块分配算法

❖ 2）、按进程大小比例分配算法：根据进程的大小按比例分配物理块的算法。

$$s = \sum_{i=1}^{n} s_i \qquad b_i = \frac{s_i}{s} \times m$$

- Si：每个进程的页面数；bi：每个进程所能分到的物理块数，b应该取整，它必须大于最小物理块数；m：系统中可用的物理块总数。

（3）物理块分配算法

3)、考虑优先权的分配算法

在实际应用中，为了照顾到重要的、紧迫的作业能尽快地完成，应为它分配较多的内存空间。

3)、考虑优先权的分配算法

通常采取的方法是把内存中可供分配的所有物理块分成两部分：一部分按比例地分配给各进程；另一部分则根据各进程的优先权，适当地增加其相应份额后，分配给各进程。

在有的系统中，如重要的实时控制系统，则可能是完全按优先权来为各进程分配其物理块的。

页面调度策略

- （1）调入时机：
  - 预调页策略：根据空间局部性，将那些预计在不久之后便会被访问的页面预先调入内存。
    - 目前：成功率≤50%
    - 主要用于进程的首次调入时，由程序员指出应该先调入哪些页。
  - 请求调页策略：缺页时，由OS将其调入内存。
    - 每次仅调入一页，较费系统开销，增加了磁盘I/O的启动频率。

（2）从何处调入页面

在请求分页系统中的外存分为两部分：用于存放文件的文件区和用于存放对换页面的对换区。

通常，由于对换区是采用连续分配方式，而文件区是采用离散分配方式，故对换区的磁盘I/O速度比文件区的高。

（2）从何处调入页面

1）对换区空间充足：

全部从对换区调入所需页面，以提高调页速度。

为此，在进程运行前，便须将与该进程有关的文件从文件区拷贝到对换区。

2）对换区空间不够：

凡是不会被修改的文件都直接从文件区调入；而当换出这些页面时，由于它们未被修改而不必再将它们换出，以后再调入时，仍从文件区直接调入。

但对于那些可能被修改的部分，在将它们换出时，便须调到对换区，以后需要时，再从对换区调入。

3）UNIX方式：

由于与进程有关的文件都放在文件区，故凡是未运行过的页面，都应从文件区调入。而对于曾经运行过但又被换出的页面，由于是被放在对换区，因此在下次调入时，应从对换区调入。

由于UNIX系统允许页面共享，因此，某进程所请求的页面有可能已被其它进程调入内存，此时也就无须再从对换区调入。

3. 页面调入过程

❖ 性能评价

- 优点
  - 有效地解决了碎片问题
  - 虚存的实现
- 缺点
  - 要求相应的硬件支持
  - 增加系统开销
  - 请求调页的算法选择不当，可能产生抖动现象
  - 没有彻底消除碎片问题。

缺页率

❖ 缺页率：

* 假设一个进程的运行过程中，访问页面成功（即所访问的页面在内存中）的次数为S，访问页面失败（即所访问的页面不在内存中，需要从外存调入）的次数为F，则该进程总的页面访问次数为A=S+F，则该进程在其运行过程中的缺页率为：

$$f = \frac{F}{A}$$

置换算法



4.6.3 页面置换算法

置换算法：把选择换出页面的算法称为页面置换算法。

目的：减少对换量，提高系统性能。

一个好的页面置换算法应具有较低的页面更换频率。
从理论上讲，应该将那些以后不会再访问的页面换出，或把那些在较长时间内不会再访问的页面调出。

最佳置换算法

1. 最佳置换算法

* 最佳置换算法所选择的被淘汰页面，将是以后永不使用的，或许是在最长(未来)时间内不再被访问的页面。
* 该算法通常可保证获得最低的缺页率。

例1： 假定系统为某进程分配了三个物理块， 并考虑有以下的页面号引用串：7，0，1，2，0，3，0，4，2，3，0，3，2，1，2，0，1，7，0，1
请画出利用最佳页面置换算法时的置换图 。

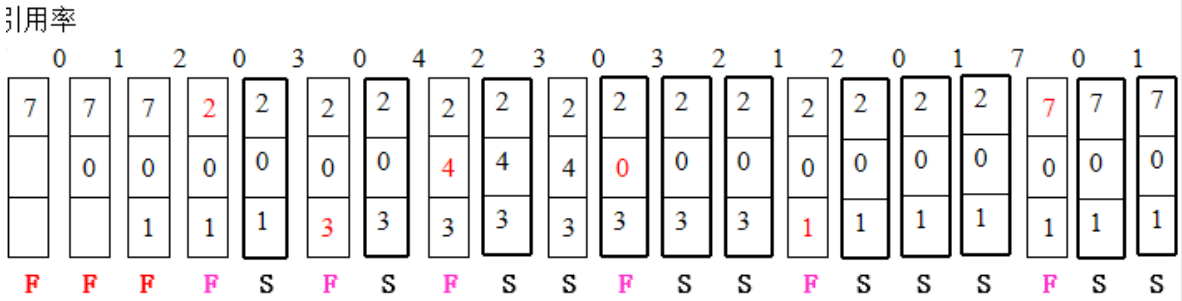


图 4-26 利用最佳页面置换算法时的置换图

采用最佳置换算法：

发生了 6 次页面置换， 9 次缺页中断，缺页率为： 9/20=0.45=45% 。

FIFO页面置换

❖ 引入：

- 最佳置换算法具有最好的性能，
- 但实际上是 无法实现 的。
- 该算法是一种理论上的算法。

❖ 最佳置换算法实际上是无法实现：

- 因为人们目前还无法预知，一个进程在内存的若干个页面中，哪一个页面是未来最长时间内不再被访问的。
- 该算法是一种理论上的算法。

❖ 先进先出页面置换算法的原理：

- 该算法总是淘汰最先进入内存的页面，
- 即选择在内存中驻留时间最久的页面予以淘汰。

❖ FIFO页面置换算法的实现：

- 只需把一个进程已调入内存的页面按先后次序链接成一个队列，并设置一个指针，称为替换指针，使它总是指向最老的页面。
- 当需要淘汰一个页面时，就选择该指针所指的页面。

例2：假定系统为某进程分配了三个物理块，并考虑有以下的页面号引用串：7，0，1，2，0，3，0，4，2，3，0，3，2，1，2，0，1，7，0，1。请画出利用FIFO页面置换算法时的置换图，并计算出缺页次数和缺页率。

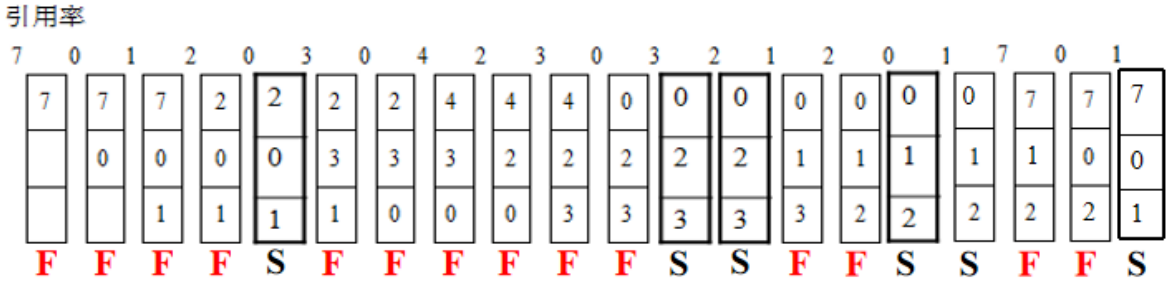


图 4-31 利用FIFO置换算法时的置换图

采用FIFO置换算法：

发生了 11 次页面置换，14 次缺页中断，缺页率为：14/20=0.7 = 70%。

❖ Belady现象：

- 采用FIFO算法时，如果对一个进程未分配它所要求的全部页面，有时就会出现分配的物理块数增多，缺页率反而提高的一种异常现象。

❖ 出现Belady现象的原因：

- FIFO算法的置换特征与进程访问内存的动态特征是矛盾的。
- 即被置换的页面并不是进程不会访问的。
- 也即是说各个页面的调入次序不能准确反映页面的访问情况，因为先进入内存的某些页面，可能是今后经常要访问的页面。

抖动

❖ 此外，FIFO页面置换算法可能会导致进程出现抖动。

❖ 抖动：

* 指系统频繁地更换页面，以致进程在运行中把大部分时间花在来回进行页面的换进/换出上，我们就称该进程发生了抖动。

❖ FIFO页面置换算法会导致进程出现抖动的原因：

* 因为在内存中驻留时间最久的页面未必是最长时间以后才被访问的页面。
* 频繁地更换页面。

❖ 小结：

FIFO页面置换算法是最直观的算法，其实现也简单。

但实际应用极少 。

❖ FIFO页面置换算法实际应用极少的原因：

* 该算法与进程实际运行的规律不相适应，因为在进程中，有些页面经常被访问，
  - 比如，含有 全局变量 常用函数 例程 、等的页面。
* FIFO算法并不能保证这些页面不被淘汰。
* 因而它可能是性能最差的算法。

LRU置换算法

3. 最近最久未使用(LRU)置换算法

（1） LRU(Least Recently Used)置换算法的描述

FIFO置换算法性能之所以较差，是因为它所依据的条件是各个页面调入内存的时间，而页面调入的先后并不能反映页面的使用情况。

最近最久未使用(LRU)页面置换算法，是根据页面调入内存后的使用情况进行决策的。

❖ （1） LRU置换算法的描述

- 根据页面调入内存后的使用情况进行决策的。
- 将“最近的过去”，作为“最近的将来”。
- 即选择最近最久未使用的页面予以淘汰。

该算法赋予每个页面一个访问字段，用来记录一个页面自上次被访问以来所经历的时间t，当须淘汰一个页面时，选择现有页面中其t值最大的，即最近最久未使用的页面予以淘汰。

[例3]：假定系统为某进程分配了三个物理块， 有以下的页面号引用串：7，0，1，2，0，3，0，4，2，3，0，3，2，1，2，0，1，7，0，1。利用LRU页面置换算法时的置换图，**并计算出缺页次数和缺页率**。

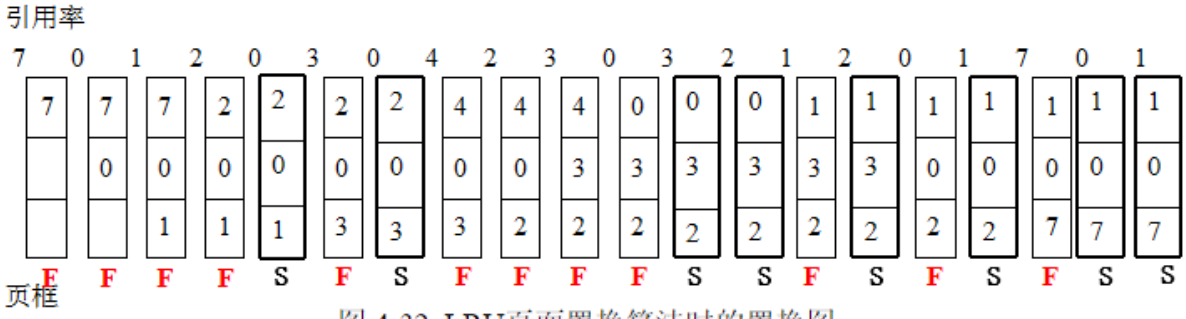


图 4-32 LRU页面置换算法时的置换图

采用最佳置换算法：

发生了 9 次页面置换，12 次缺页中断，缺页率为：12/20=0.6=60% 。

练习：假定系统为某进程分配了三个物理块，并考虑有以下的页面号引用串：2, 3, 2, 1, 5, 2, 4, 5, 3, 2, 5, 2. 请画出利用LRU页面置换算法时的置换图，并计算出缺页次数和缺页率。

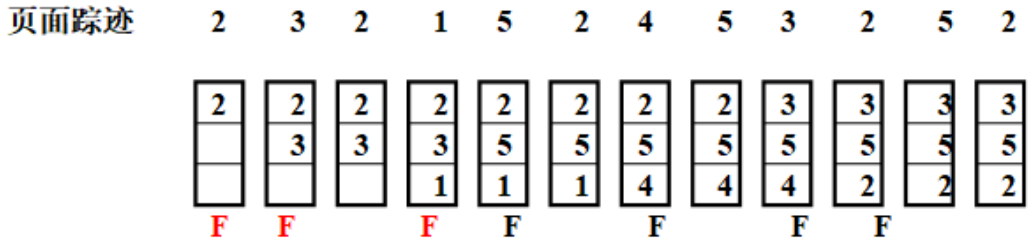


图 4-33 LRU页面置换算法时的置换图

采用最佳置换算法：

发生了 4 次页面置换，7 次缺页中断，缺页率为：7/12=0.58=58%。

❖ （2）LRU算法的硬件支持：（用来记录谁最近最久未访问）

* 两类硬件之一的支持：寄存器或栈。
* 1）寄存器：（定时右移）

▪ 为了记录某进程在内存中各页的使用情况，须为每个在内存中的页面配置一个移位寄存器，可表示为：

$$R = R_{n-1}R_{n-2}R_{n-3} \cdots R_2R_1R_0$$

❖ （2）LRU算法的硬件支持：（用来记录谁最近最久未访问）

* 1）寄存器：（定时右移）

▪ 当进程访问某个物理块时，要将相应寄存器的R_{n-1}位置为1.

▪ 定时信号将每隔一定时间将寄存器右移一位。

▪ 如果把n位寄存器的数看做是一个整数，那么，具有最小数值的寄存器所对应的页面，就是最近最久未使用的页面。

实页 \ R	R_7	R_6	R_5	R_4	R_3	R_2	R_1	R_0
1	0	1	0	1	0	0	1	0
2	1	0	1	0	1	1	0	0
3	0	0	0	0	0	1	0	0
4	0	1	1	0	1	0	1	1
5	1	1	0	1	0	1	1	0
6	0	0	1	0	1	0	1	1
7	0	0	0	0	0	1	1	1
8	0	1	1	0	1	1	0	1

图 4-34 某进程具有8个页面时的LRU访问情况

由图可以看出，第3个内存页面的R值最小，当发生缺页时，首先将它置换出去。

- （2）LRU算法的硬件支持：（用来记录谁最近最久未访问）
  - 2）栈：
    - 当进程访问某页时，将其移出压入“栈顶”，置换时将“栈底”页面换出。

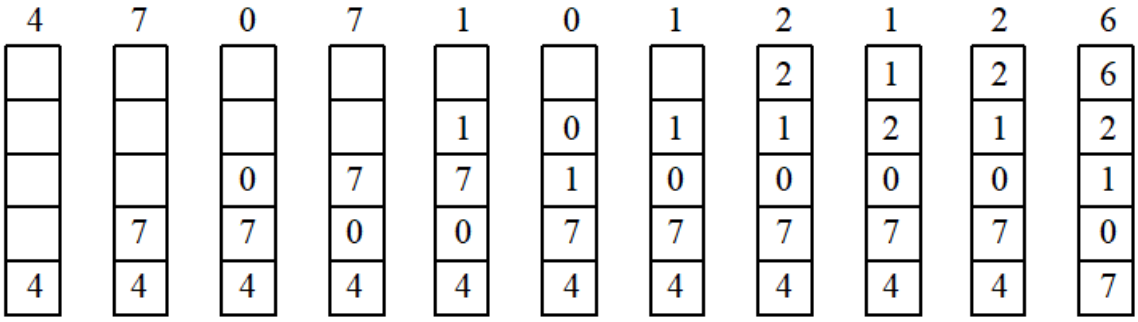


图4-35 用栈保存当前使用页面时栈的变化情况

请求分页管理方式

内存有效访问时间

- 请求分页管理方式中，内存有效访问时间（EAT）的构成：
  - 访问页表的时间
  - 访问实际物理地址数据的时间
  - 缺页中断的处理时间

- 具有快表机制的请求分页管理方式中：

- （1）被访问页在内存中，且其对应的页表项在快表中。

$$EAT = \lambda + t$$

  - λ 为查找快表的时间，t 为访问实际物理地址所需要的时间

- 具有快表机制的请求分页管理方式中：

- （2）被访问页在内存中，且其对应的页表项不在快表中。

$$EAT = \lambda + t + \lambda + t = 2 \times (\lambda + t)$$

  - λ 为查找快表的时间，t 为访问实际物理地址所需要的时间

- 具有快表机制的请求分页管理方式中：
- （3）被访问页不在内存中。
  - EAT包括：
    - 查找快表的时间、查找页表的时间、处理缺页中断的时间、更新快表的时间和访问实际物理地址的时间。

$$EAT = \lambda + t + \varepsilon + \lambda + t = \varepsilon + 2 \times (\lambda + t)$$

  - ε 为缺页中断处理时间。

- 考虑快表的命中率和缺页率等因素：

$$EAT = \lambda + a \times t + (1-a) \times [t + f \times (\varepsilon + \lambda + t) + (1-f) \times (\lambda + t)]$$

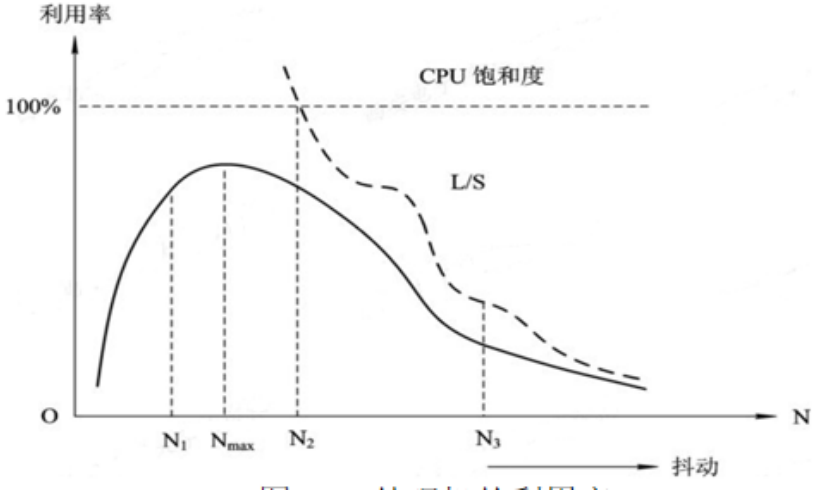
- 如果不考虑命中率，仅考虑缺页率，有：

$$EAT = t + f \times (\varepsilon + t) + (1-f) \times t$$

系统抖动

❖ 1. 多道程序度与处理机的利用率

由于虚拟存储器系统能从逻辑上扩大内存，这时，只需装入一个进程的部分程序和数据便可开始运行，故人们希望在系统中能运行更多的进程，即增加多道程序度，以提高处理机的利用率。但处理机的实际利用率却如图4-39中的实线所示。



2. 产生“抖动”的原因

❖ 发生“抖动”的根本原因是，同时在系统中运行的进程太多，由此分配给每一个进程的物理块太少，不能满足进程正常运行的基本要求，致使每个进程在运行时，频繁地出现缺页，必须请求系统将所缺之页调入内存。这会使得在系统中排队等待页面调进/调出的进程数目增加。

❖ 显然，对磁盘的有效访问时间也随之急剧增加，造成每个进程的大部分时间都用于页面的换进/换出，而几乎不能再去做任何有效的工作，从而导致发生处理机的利用率急剧下降并趋于0的情况。我们称此时的进程是处于“抖动”状态。

❖ 进程发生缺页率的时间间隔与进程所获得的物理块数有关。图4-40示出了缺页率与物理块数之间的关系。

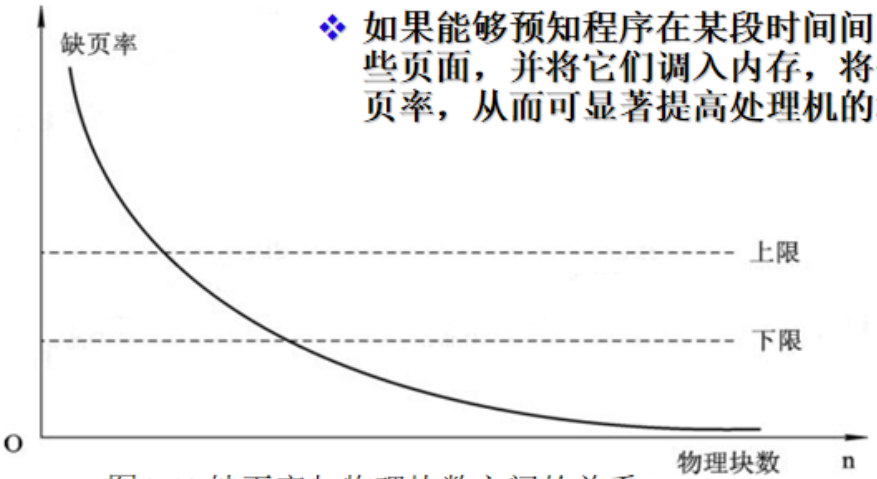


图4-40 缺页率与物理块数之间的关系

❖ 如果能够预知程序在某段时间间隔内要访问哪些页面，并将它们调入内存，将会大大降低缺页率，从而可显著提高处理机的利用率。

请求分段管理方式（未完）

设备管理

IO层次结构

5.1.2 I/O系统的层次结构和模型

1. I/O软件的层次结构

通常把I/O 软件组织成四个层次，如图5-1所示。

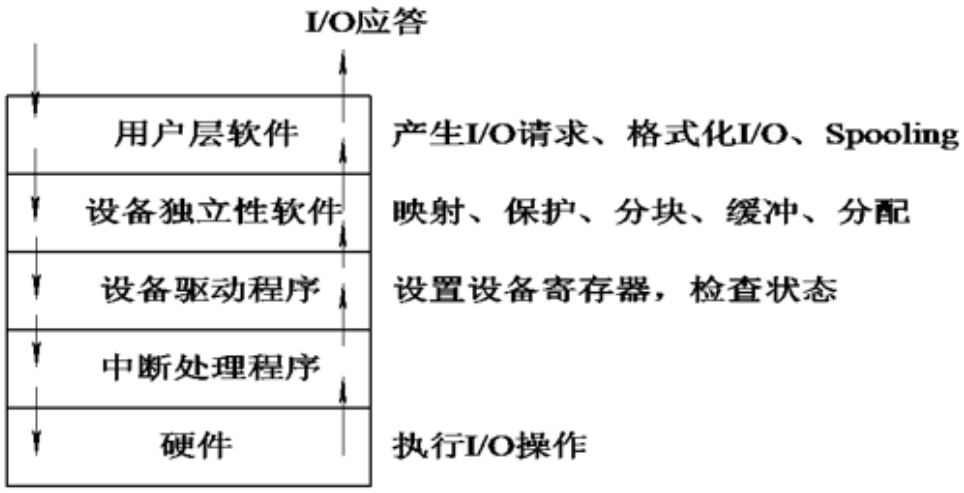


图5-1 I/O系统的层次结构

2. I/O系统中各种模块之间的层次视图

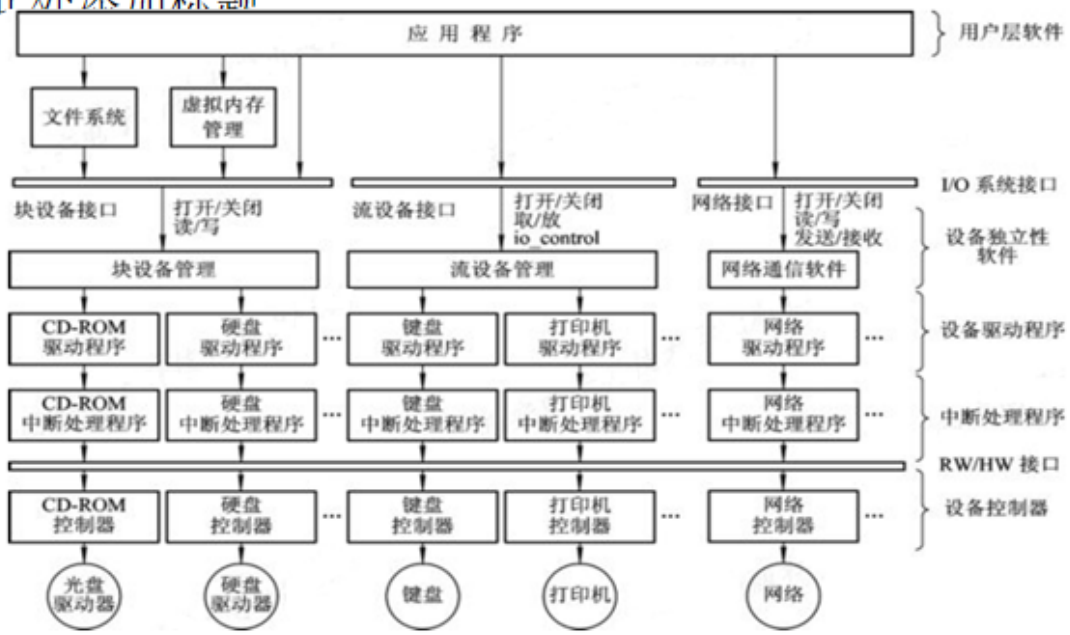
为了能更清晰地描述I/O系统中主要模块之间的关系，I/O系统中各种I/O模块之间的层次视图如图5-2所示。

1) I/O系统的上、下接口

(1) I/O系统接口。

(2) 软件/硬件(RW/HW)接口。

击此处添加标题



2) I/O系统的分层

与前面所述的I/O软件组织的层次结构相对应，I/O系统本身也可分为如下三个层次：

(1) 中断处理程序。

(2) 设备驱动程序。

(3) 设备独立性软件。

5.1.3 I/O系统接口

在I/O系统与高层之间的接口中，根据设备类型的不同，又进一步分为若干个接口。在图5-2中示出了块设备接口、流设备接口和网络接口。

1. 块设备接口

(1) 块设备。

(2) 隐藏了磁盘的二维结构。

(3) 将抽象命令映射为低层操作。

5.1.3 I/O系统接口

在I/O系统与高层之间的接口中，根据设备类型的不同，又进一步分为若干个接口。在图5-2中示出了块设备接口、流设备接口和网络接口。

1. 块设备接口

(1) 块设备。

(2) 隐藏了磁盘的二维结构。

(3) 将抽象命令映射为低层操作。

2. 流设备接口

流设备接口是流设备管理程序与高层之间的接口。该接口又称为字符设备接口，它反映了大部分字符设备的本质特征，用于控制字符设备的输入或输出。

(1) 字符设备。

(2) get和put操作。

(3) in-control指令。

3. 网络通信接口

在现代OS中，都提供了面向网络的功能。但首先还需要通过某种方式把计算机连接到网络上。同时操作系统也必须提供相应的网络软件和网络通信接口，使计算机能通过网络与网络上的其它计算机进行通信或上网浏览。

IO设备和设备控制器

5.2 I/O设备和设备控制器

I/O设备一般是由执行I/O操作的**机械部分和执行控制I/O的电子部件**组成。通常将这两部分分开，执行I/O操作的机械部分就是**一般的I/O设备**，而执行控制I/O的电子部件则称为**设备控制器或适配器(adapter)**。

在微型机和小型机中的控制器常做成印刷电路卡形式，因而也常称为控制卡、接口卡或网卡，可将它插入计算机的扩展槽中。

在有的大、中型计算机系统中，还配置了I/O通道或I/O处理机。

类型

5.2.1 I/O设备

1. I/O设备的类型

1) 按设备的使用特性分类

- **存储设备**，也称外存、辅助存储器：用以存储信息的主要设备。该类设备存取速度较内存慢，但容量比内存大得多，相对价格也便宜。
- **输入/输出设备**，又具体可分为**输入设备、输出设备和交互式设备。**

1) 按设备的使用特性分类

- 输入/输出设备：
  - **输入设备：**接收外部信息，如键盘、鼠标、扫描仪、视频摄像、各类传感器等。
  - **输出设备：**是用于将计算机加工处理后的信息送向外部的设备，如**打印机、绘图仪、音响输出设备**等。
  - **交互式设备**则是集成上述两类设备，主要是**显示器**，用于同步显示用户命令以及命令执行的结果。

1. I/O设备的类型

2) 按传输速率分类

●**低速设备**，其传输速率仅为每秒钟几个字节至数百个字节的一类设备。如：键盘、鼠标器等设备。

●**中速设备**，其传输速率在每秒钟数千个字节至数十万个字节的一类设备。如：行式打印机、激光打印机等。

●**高速设备**，其传输速率在数十万字节至千兆字节的一类设备。如：磁带机、磁盘机、光盘机等。

3) 按信息交换的单位分类

●**块设备**，这类设备用于存储信息。信息的存取总是以数据块为单位。它属于有结构设备。典型的块设备是磁盘，其基本特征是 a)传输速率较高，通常每秒钟为几兆位；b)可寻址，即对它可随机地读/写任一块；此外，磁盘设备的I/O常采用DMA方式。

●**字符设备**，用于数据的输入和输出。其基本单位是字符，故称为字符设备。如：打印机、串口

4）从设备的共享属性上，可将系统中的设备分为：

(1) 独占设备，进程应互斥地访问这类设备，即系统一旦把这类设备分配给了某进程后，便由该进程独占，直至用完释放。典型的独占设备有**打印机、磁带机**等。系统在对独占设备进行分配时，还应考虑到分配的安全性。

(2) 共享设备，是指在一段时间内允许多个进程同时访问的设备。典型的共享设备是**磁盘**，当有多个进程需对磁盘执行读、写操作时，可以交叉进行，不会影响到读、写的正确性。

IO通道*

5.2.4 I/O通道

1. I/O通道设备的引入

虽然在CPU与I/O设备之间增加了设备控制器后，已能大大减少CPU对I/O的干预，但当主机所配置的外设很多时，CPU的负担仍然很重。为此，在CPU和设备控制器之间又增设了I/O通道(I/O Channel)。

I/O通道：

一种特殊的执行I/O指令的处理机，并通过执行通道(I/O)程序来控制I/O操作，与CPU共享内存，可以有自己的总线。

2. 通道类型

1) 字节多路通道(Byte Multiplexor Channel)

这是一种按字节交叉方式工作的通道。它通常都含有许多非分配型子通道，其数量可从几十到数百个，每一个子通道连接一台I/O设备，并控制该设备的I/O操作。这些子通道按时间片轮转方式共享主通道。

单击此处添加标题

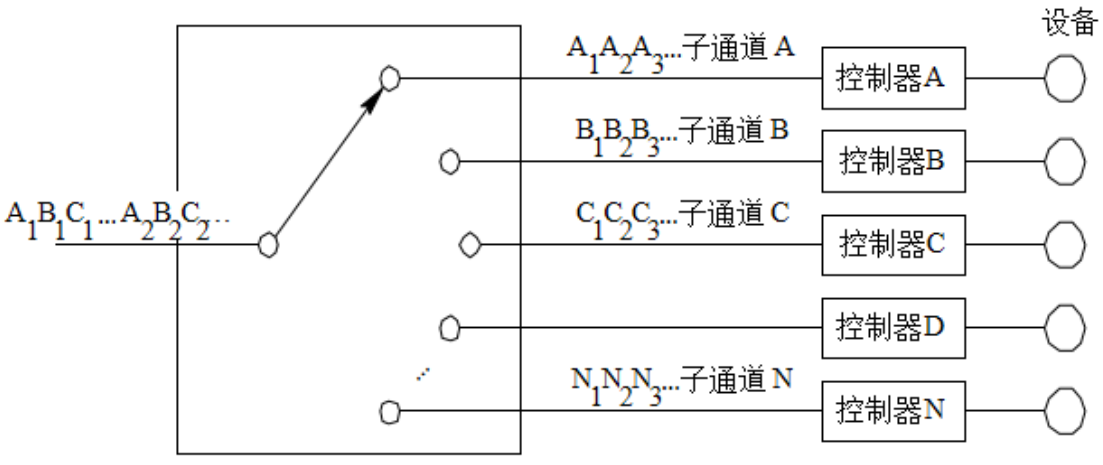


图5-6 字节多路通道的工作原理

2) 数组选择通道(Block Selector Channel)

字节多路通道不适于连接高速设备，这推动了按数组方式进行数据传送的数组选择通道的形成。

该通道可以**连接多台高速设备**。

但它只含有**一个分配型子通道**，在**一段时间内只能执行一道通道程序，控制一台设备进行数据传送**，致使当某台设备占用了该通道后，便一直由它**独占**，直至该设备传送完毕释放该通道。

这种通道的利用率很低。

3) 数组多路通道(Block Multiplexor Channel)

数组选择通道虽有很高的传输速率，**但它却每次只允许一个设备传输数据。**数组多路通道是将数组选择通道传输速率高和字节多路通道能使各子通道(设备)分时并行操作的优点相结合而形成的一种新通道。

3. “瓶颈”问题

由于通道价格昂贵，致使机器中所设置的通道数量势必较少，这往往又使它成了I/O的瓶颈，进而造成整个系统吞吐量的下降。

中断处理程序（重点）（中断和陷入）

5.3 中断机构和中断处理程序

中断在操作系统中有着特殊重要的地位，它是多道程序得以实现的基础，没有中断，就不可能实现多道程序，因为**进程之间的切换是通过中断来完成的**。另一方面，**中断也是设备管理的基础**，为了提高处理机的利用率和实现CPU与I/O设备并行执行，也必需有中断的支持。

中断处理程序是I/O系统中最低的一层，它是整个I/O系统的基础。

5.3.1 中断简介

1. 中断和陷入

1) 中断：

中断是指CPU对I/O设备发出来的中断信号的一种响应。由于中断是由外部设备引起的，故又称为外中断。

2) 陷入

由CPU内部事件所引起的中断，如进程在运算中发生了上溢或下溢，如程序出错，如非法指令、地址越界、以及电源故障等。把这类中断称为内中断或陷入。